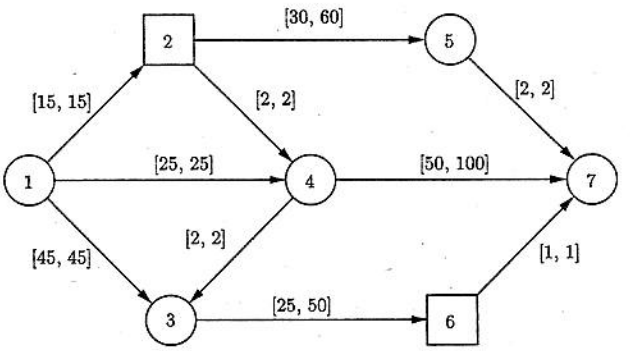


图 2.13 多商品网络流：示例网络

假设我们考虑 2 个商品流，分别表示如下：

（1）commodity 1：[1, 7, 25]；

（2）commodity 2：[2, 6, 2]；

其中，[1, 7, 25] 表示起点为 1、终点为 7、需求是 25 单位。那么该问题的最优解如图 2.14 所示，总成本为 1873。

下面我们给出 MCNF 的数学模型。首先，引入决策变量 x_{ij}^k，表示在弧 (i,j) 上商品流 k 的流量，则 MCNF 的模型为

$$\min \quad \sum_{(i,j)\in A}\sum_{k} c_{ij}^k x_{ij}^k \tag{2.43}$$

$$\text{s.t.} \quad \sum_{j} x_{ij}^k - \sum_{i} x_{ji}^k = \begin{cases} d_k, & \text{如果 } i = s_k,\ k \in K \\ -d_k, & \text{如果 } i = t_k,\ k \in K \\ 0, & \text{其他, } \forall i \in V, k \in K,\ i \neq s_k,\ i \neq t_k \end{cases} \tag{2.44}$$

$$\sum_{k} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{2.45}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A, k \in K \tag{2.46}$$

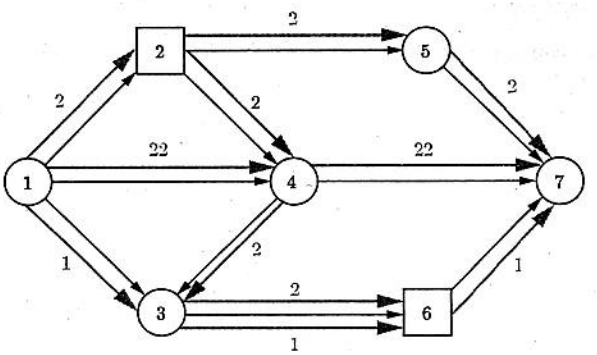


图 2.14 多商品网络流：最优解

在 MCNF 中，决策变量 x_{ij}^k 是连续变量，但是在一些场景中，该决策变量也有可能是整数变量。在本章不做展开。

MCNF 的模型还能有另外一种形式（Wang, 2018）。为了更为方便地查看，我们将上述算例的具体形式写在 Excel 表格里。如图 2.15 所示。

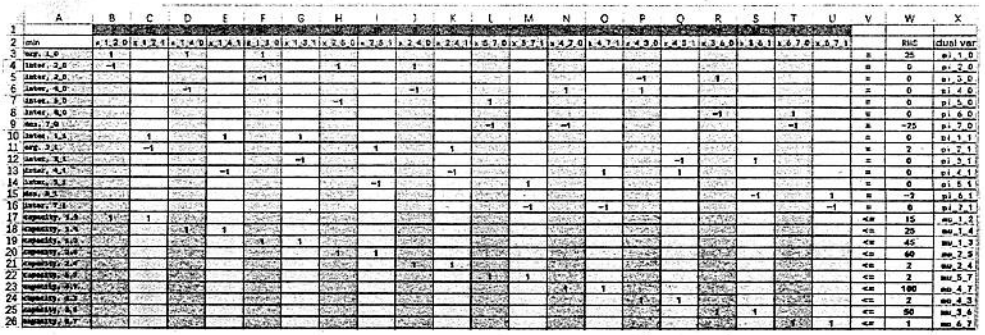


图 2.15　MCNF 模型的表格形式

由于约束 (2.44) 是满足 $\forall i \in V, k \in K$ 条件的，因此我们可以按照商品流（commodity）的序号 k，将约束矩阵分解成 K 个方块，依次重新排列开。也就是将图 2.15 中的表格按照商品流 k 把上面的表格整一下，变成更直观的形式，如图 2.16 所示。

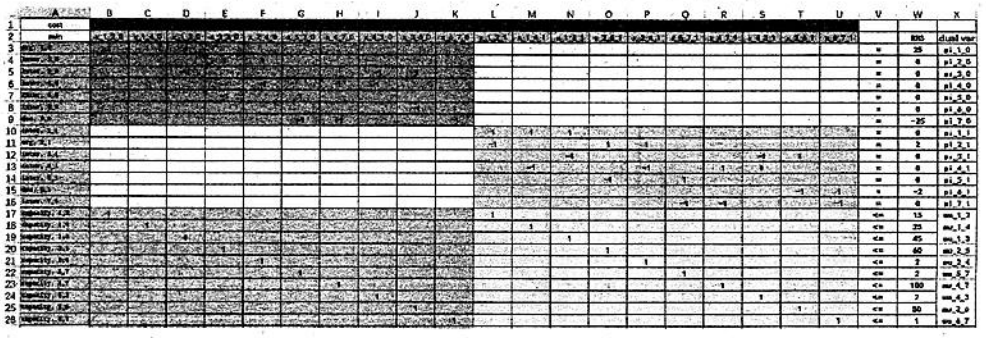


图 2.16　MCNF 模型的表格形式（排序后）

上面的模型符合如下形式：

$$\begin{bmatrix} \widetilde{\boldsymbol{N}} & \\ & \widetilde{\boldsymbol{N}} \\ \boldsymbol{I} & \boldsymbol{I} \end{bmatrix} \cdot \begin{bmatrix} \boldsymbol{X}^1 \\ \boldsymbol{X}^2 \end{bmatrix} \begin{array}{ccc} = & \boldsymbol{b}_1 & = [25,0,0,0,0,0,-25]^{\mathrm{T}} \\ = & \boldsymbol{b}_2 & = [0,2,0,0,0,-2,0]^{\mathrm{T}} \\ \leqslant & \boldsymbol{u} & = [15,25,45,60,2,2,100,2,50,1]^{\mathrm{T}} \end{array}$$

其中，$\boldsymbol{X}^1$ 和 $\boldsymbol{X}^2$ 分别代表商品流 1 和商品流 2 对应的决策变量。结合图 2.16，由于约束 (2.44) 中有 $\forall i \in V$，因此每一个分块矩阵 $\widetilde{\boldsymbol{N}}$ 就对应 $\forall i \in V$，也就是 $|V|$ 行。而最后的关于容量（capacity）的约束中，约束是针对 $\forall (i,j) \in A$ 的，而且每一行都是流经这一条弧

上的所有流量的总和，因此矩阵的元素全是 1，所以约束矩阵就对应一个单位矩阵 $\boldsymbol{I}$，而且每个商品流都对应一个单位矩阵 $\boldsymbol{I}$。

综合以上，我们可以给出 MCNF 的另外一种建模方法，叫作无 OD 对的建模（formulation without OD pairs）。具体模型为

$$\min \sum_k (\boldsymbol{c}^k)^{\mathrm{T}} \boldsymbol{x}^k \tag{2.47}$$

$$\text{s.t.} \sum_k x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{2.48}$$

$$\boldsymbol{N}\boldsymbol{x}^k = \boldsymbol{b}^k, \qquad \forall\, k \in K \tag{2.49}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A, k \in K \tag{2.50}$$

在上述的建模方式中，约束 (2.49) 其实做了简化，即令 $\boldsymbol{x}_{ij}^k = \boldsymbol{x}^k, \forall (i,j) \in A, k \in K$ 这一步省去了流平衡的约束，但是实质上模型是没有任何变化的，只是从形式上来讲，更直观一些。

当流量可以为小数时，MCNF 不是 NP-hard 问题。因为流量可以为小数时，MCNF 的模型是一个线性规划，可以用求解器直接很快地求解。另外，流量可以为小数的 MCNF，也存在一些复杂度非常低的、高效的多项式时间近似算法，这里列举一篇参考文献（Garg and Könemann，1998），更详细的文献请读者自行查找学习。

但是当流量必须为整数时，MCNF 就变成了 NP-complete 问题（当然也是 NP-hard 问题）[①]。即使只有 2 个商品流、边的容量都为 1 的情况下，MCNF 都是 NP-complete 问题，因此该问题被标记为强 NP-complete 问题（Even et al., 1975）。

2.6 多商品流运输问题

在 MCNF 中，若商品流要求直接从出发点配送到目的地时，MCNF 就退化成多商品流运输问题（Multi-commodity Transportation Problem, MCTP）。换句话说，MCNF 和 MCTP 的区别在于，MCTP 中商品流（货物）并没有网络中间节点间的转运，而是直接从供应商点运输到客户点。

MCTP 可以描述为如下问题。假设供应商的集合为 S，客户点的集合为 C，网络中的弧段的集合为 $A = \{(i,j) | i \in S, j \in C\}$，商品流的集合为 K，供应商 $i \in S$ 供应商品 k 的量为 s_i^k，客户 $j \in C$ 对商品 k 的需求量为 d_j^k，将商品 k 从供应商 i 运输到客户 j 的成本为 c_{ij}^k，网络中弧段 (i,j) 的容量为 u_{ij}。MCTP 的目标就是以最小的成本，将货物从供应商点配送给顾客，以满足所有客户的需求，同时满足供需关系和容量约束。基于上述描述，我们引入决策变量 x_{ij}^k，表示网络中弧段 (i,j) 上商品流 k 的流量。则 MCTP 可以建模为下面的线性规划：

① https://en.wikipedia.org/wiki/Multi-commodity_flow_problem.

$$\min \quad \sum_{k\in K}\sum_{(i,j)\in A} c_{ij}^k x_{ij}^k \tag{2.51}$$

$$\text{s.t.} \quad \sum_{j\in C} x_{ij}^k = s_i^k \qquad \forall i \in S,\ \forall k \in K \tag{2.52}$$

$$\sum_{i\in S} x_{ij}^k = d_j^k, \qquad \forall j \in C,\ \forall k \in K \tag{2.53}$$

$$\sum_{k\in K} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{2.54}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A,\ \forall k \in K \tag{2.55}$$

关于 MCTP，本书会在 Dantzig-Wolfe 分解算法部分再做详细探讨，本节仅给出其数学模型。

当不考虑容量约束，即去掉约束 (2.54)，并且仅有一种商品时，MCTP 就退化成了运输问题（Transportation Problem, TP）。运输问题的线性规划模型为

$$\min \quad \sum_{(i,j)\in A} c_{ij} x_{ij} \tag{2.56}$$

$$\text{s.t.} \quad \sum_{j\in C} x_{ij} = s_i, \qquad \forall i \in S \tag{2.57}$$

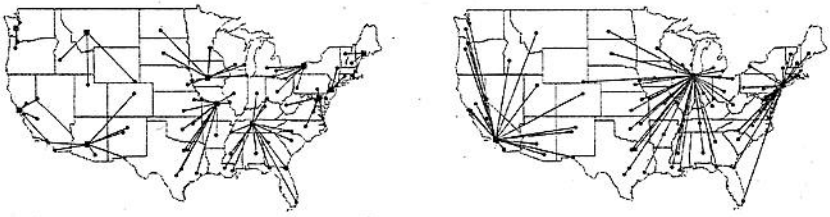
$$\sum_{i\in S} x_{ij} = d_j, \qquad \forall j \in C \tag{2.58}$$

$$x_{ij} \geqslant 0, \qquad \forall (i,j) \in A \tag{2.59}$$

关于运输问题，还有一种拓展，即固定费用运输问题（Fixed-charge Transportation Problem, FCTP）。在固定费用运输问题中，每条弧段如果有货物运输，即视为开通。开通每条弧段都会产生相应的一次性固定成本。关于该问题更详细的探讨，见 Benders 分解算法相关章节。

2.7 设施选址问题

设施选址问题（Facility Location Problem, FLP）是一类非常重要的运筹优化问题。在实际生产中，企业面临的主要战略决策之一就是设施的数量和位置，如仓库、零售店或其他实体设施等。这些相关问题称为设施选址问题。其关键就是如何在建造设施的成本和顾客的服务质量之间做好权衡（Snyder and Shen, 2019）。如果开通很多设施，如图 2.17(a) 所示，则会产生高昂的设施成本（建造成本和维护成本），但我们可以提供高质量的服务，因为大多数客户都靠近工厂。如果我们仅开通少量的设施，如图 2.17(b) 所示，则会大幅降低设施成本，但顾客距离工厂较远。大多数选址问题都需要做两组相关的决策：① 在哪里建造设施？② 将哪些客户分配给哪些设施去服务？因此，设施选址问题有时也称为选址-分配问题（Location-allocation Problem）。



(a) 开通大量设施　　(b) 开通少量设施

注：方块代表设施，圆点代表顾客（Snyder and Shen, 2019）

图 2.17 不同设施选址决策

设施选址问题的一个最基础的版本就是无容量限制的设施选址问题（Uncapacitated Facility Location Problem, UFLP）。首先，我们给出 UFLP 的问题描述：给定一个可选设施点的集合 $N=\{1,2,\cdots,n\}$ 和一个客户点的集合 $M=\{1,2,\cdots,m\}$。假设每一个设施点都对应固定成本 f_j，即如果设施 j 被开通，则会产生固定成本 f_j。并且，如果顾客 i 的需求被设施 j 满足，则会产生相应的运输成本 c_{ij}。UFLP 就是要决策开通哪些设施点，使得所有客户的需求都被满足，并且要最小化总的设施建设成本和服务成本（Wolsey, 1998）。

注意：如果不考虑运输成本，UFLP 与覆盖问题（Covering Problem）就非常相似。

基于上述描述，引入下面的决策变量：

（1）y_j：如果设施 $j\in N$ 被开通，则 $y_j=1$，否则 $y_j=0$；

（2）x_{ij}：顾客点 i 的需求被设施 j 满足的比例。

下面我们给出 UFLP 的模型：

$$\min\ z=\sum_{i\in M}\sum_{j\in N}c_{ij}x_{ij}+\sum_{j\in N}f_jy_j \tag{2.60}$$

$$\text{s.t.}\quad \sum_{j\in N}x_{ij}=1, \qquad \forall i\in M \tag{2.61}$$

$$x_{ij}\leqslant y_j, \qquad \forall i\in M,\forall j\in N \tag{2.62}$$

$$\boldsymbol{x}\in\mathbb{R}^{|M|\times|N|},\boldsymbol{y}\in\mathbb{B}^{|N|} \tag{2.63}$$

关于 UFLP，还有另外一种提法，即 Uncapacitated Fixed-charge Location Problem。为了方便起见，本书中统一使用第一种提法。

UFLP 可以使用拉格朗日松弛算法提高求解效率，详见本书拉格朗日松弛相关章节。另外，文献（Snyder and Shen, 2019）第 8 章给出了使用拉格朗日松弛求解 UFLP 的更多细节，感兴趣的读者可以前往阅读。

2.8 旅行商问题

旅行商问题或者货郎担问题（Traveling Salesman Problem 或者 Traveling Salesperson Problem, TSP），是一个著名的 NP-hard 问题。TSP 的描述如下：给定一个有向图 $G=$

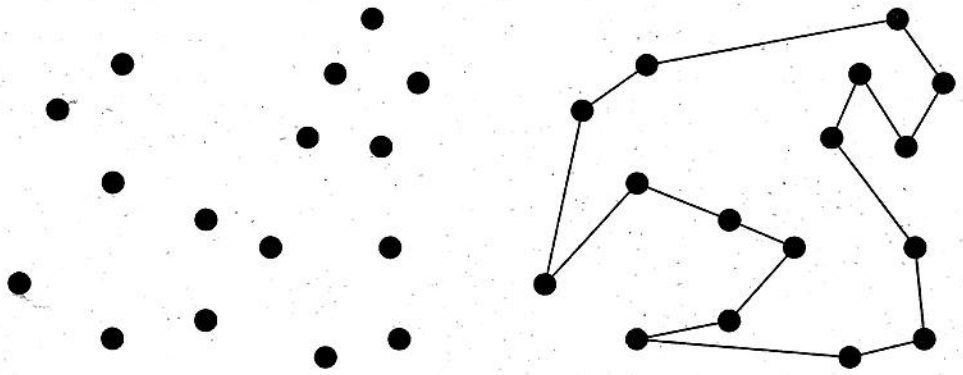
(V,E), 其中 V 是图中节点集合（$|V|=N$），E 为图中边的集合（当从点 i 到点 j 和从点 j 到点 i 的距离相等时我们将其称为 Symmetric TSP）。一般我们考虑图 G 是完全图（也就是任意两个点是可以直接到达的）。TSP 的目标就是，找到一条从起点出发（起点可以是任意点），依次不重复地经过所有其他节点，最终返回到起点的最短路径，如图 2.18(a) 所示。

也就是说，TSP 的输入是一系列坐标点的集合，输出是一个访问序列。该序列需要满足下面几个要求：

（1）每一个点都被访问且只被访问一次；

（2）每一个点都被离开且只被离开一次；

也就是说，每一个点都被访问一次，并且，经过一个点就必须离开这个点。最终，我们需要得到如图 2.18(b) 中展示的一条依次不重复经过所有节点的封闭的完美路径。



（a）Graph　　（b）Optimal solution of TSP

图 2.18　TSP：一个例子

下面我们给出 TSP 的数学模型。同样地，我们首先引入如下 0-1 决策变量 x_{ij}：

$$x_{ij}=\begin{cases}1, & \text{如果边 } (i,j) \text{ 在最优解中被选中}\\ 0, & \text{其他}\end{cases}$$

直觉上，我们可以将 TSP 建模为下面的整数规划模型：

$$\min \quad \sum_i \sum_j c_{ij}x_{ij} \tag{2.64}$$

$$\text{s.t.} \quad \sum_{i\in V} x_{ij}=1, \qquad \forall j\in V, i\neq j \tag{2.65}$$

$$\sum_{j\in V} x_{ij}=1, \qquad \forall i\in V, i\neq j \tag{2.66}$$

$$x_{ij}\in\{0,1\}, \qquad \forall i,j\in V, i\neq j \tag{2.67}$$

在上面的模型中，满足以下要求：

（1）约束 (2.65)：保证了每一个点都被访问且只被访问一次；

（2）约束 (2.66)：保证了每一个点都被离开且只被离开一次。

也就是说，上面的两个约束联合起来，可以保证，获得的解一定满足每个点都被访问一次，并且，经过一个点就会离开一个点。直觉上是可以得到一条图 2.18 中展示的路径的，即依次不重复经过所有节点的封闭的完美路径。但是不然。

也就是说，上述模型并不正确，会导致一个叫作子环路（subtour）的问题出现，如图2.19所示。

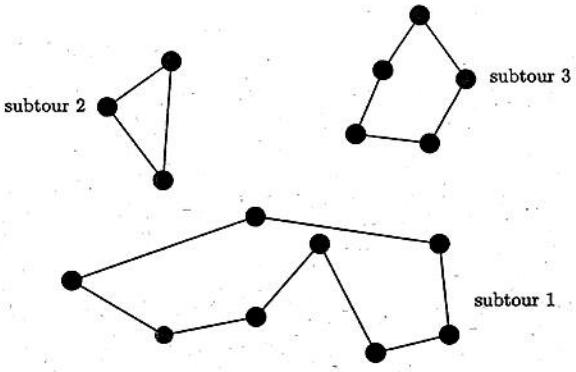


图 2.19 子环路

所谓子环路，就是没有包含所有节点 V 的一条闭环。子环路首先是一个封闭的环；其次，这个环中被访问的节点集合（假设为 S）是所有节点集合 V 的一个真子集，即 $S \subseteq V$，或 $S \subset V$ 且 $|V| < N$。如果上述模型的解出现了子环路，那么为了满足模型的约束 (2.65) 和约束 (2.66)，解中必然至少存在一个其他环路。这就导致与 TSP 想要得到的单环的解矛盾。如图 2.19 所示的情况中，图中出现了 3 个子环路。

可以看到，子环路也完美地满足：（1）每个点只被访问一次；（2）经过一个点就离开那个点。但是这样的解会导致解中含有多个相离的环。而我们需要的解是一个单个的经过所有点的大环。为了得到一个大环，就要添加消除子环路的约束来完善 TSP 的模型。

这里比较常见的消除子环路的办法有以下 2 种：

（1）加入 subtour-elimination 约束；

（2）使用 Miller-Tucker-Zemlin（MTZ）建模方法。

2.8.1 TSP 建模方法 1：子环路消除约束

之前，我们的问题描述中提到，有向图 G 中点的个数为 N，即 $|V| = N$。subtour-elimination 的思路比较直观，主要想法就是，根据子环路的特点，在模型中添加相应的约束，将其破开，通俗地讲，就是破圈。

举个例子，假如考虑一个由图中 3 个点 A,B,C 组成的子点集 $S=\{A,B,C\}$，假定它们构成了一个环 $A\to B\to C\to A$。那么这个环出现，就导致问题的“解”必须要满足：

$$x_{AB}=x_{BC}=x_{CA}=1$$

换句话说，对于点 A,B,C 组成的节点子集合 $S=\{A,B,C\}$ 而言，必须有

$$x_{AB}+x_{BC}+x_{CA}\geqslant 3$$

只有上面这个条件成立，才会导致子环路的出现。那么这个子环路不存在的条件（即破圈的方法）就是加入下面的约束：

$$x_{AB}+x_{BC}+x_{CA}<3$$

也就是说，我们只允许所有点都被包含进来的环存在，即包含点的个数为 N 的环，删除其余所有的环。那怎么做呢？一个简单的做法就是枚举，也就是我们在 TSP 中经常看到的约束：

$$\sum_{i,j\in S}x_{ij}\leqslant |S|-1,\qquad 2\leqslant |S|\leqslant n-1,S\subseteq V \tag{2.68}$$

因此，完整的 TSP 模型为

$$\min\quad \sum_{i}\sum_{j}c_{ij}x_{ij} \tag{2.69}$$

$$\text{s.t.}\quad \sum_{i\in V}x_{ij}=1,\qquad \forall j\in V,i\neq j \tag{2.70}$$

$$\sum_{j\in V}x_{ij}=1,\qquad \forall i\in V,i\neq j \tag{2.71}$$

$$\sum_{i,j\in S}x_{ij}\leqslant |S|-1,\qquad 2\leqslant |S|\leqslant n-1,\quad S\subseteq V \tag{2.72}$$

$$x_{ij}\in\{0,1\},\qquad \forall i,j\in V,i\neq j \tag{2.73}$$

subtour-elimination 的建模虽然直观且便于理解，但是对应的约束 (2.72) 其实至多有 2^N 量级的约束，略显繁杂，如果说在模型建立时就去穷举所有的约束，显然不现实。因此，在实际的代码实现中，一般是借用求解器的 callback 函数，以惰性（lazy）的方式去添加这一条约束的。

第 7 章提供了使用 Python 调用 Gurobi 求解 TSP，并用 callback 函数消除子环路的完整代码。

2.8.2 TSP 建模方法 2：MTZ 约束消除子环路

子环路的消除，除了上面的方法，还有另一种更为简捷的方法，这种写法在实战中可以避免使用 callback 函数，并且求解效率也非常不错，那就是使用著名的 Miller-Tucker-Zemlin（MTZ）约束（Miller et al., 1960）。该方法是引入一个决策变量 μ_i，满足如下要求：

（1）每个节点都对应一个决策变量 μ_i；

（2）利用 μ_i 构造 Miller-Tucker-Zemlin 约束，加入 TSP 的数学模型中。

这样就可以完美地解决子环路的问题。

具体来讲，就是引入 $|N|$ 个辅助决策变量 $\mu_i, \forall i \in V, \mu_i \geqslant 0$，然后对于每条边 $(i,j) \in E$，构造下面的 MTZ 约束：

$$\mu_i - \mu_j + M x_{ij} \leqslant M - 1, \quad \forall i,j \in V, i,j \neq 0, i \neq j \tag{2.74}$$

其中，M 是一个很大的正数。

> μ_i 可以理解为点 $i \in V$ 被访问的次序。比如 $\mu_1 = 5$，可以理解为点 1 是从出发点开始第 5 个被访问到的点。很多最近的论文里也是这么解释的。μ_i 的取值范围一般设置成 $\mu_i \geqslant 0$。

理论上讲，M 应当是 $\mu_i - \mu_j + 1$ 的一个上界就可以，有文献指出，取最紧的上界，效果会好一些，参见文献（Desaulniers et al., 2006），因此我们取 $M = N$。这样一来，上述约束可以拉紧为

$$\mu_i - \mu_j + N x_{ij} \leqslant N - 1, \quad \forall i,j \in V, i,j \neq 0, i \neq j \tag{2.75}$$

我们还是整理成逻辑约束的形式：

$$\mu_i - \mu_j + 1 - N(1 - x_{ij}) \leqslant 0, \quad \forall i,j \in V, i,j \neq 0, i \neq j \tag{2.76}$$

其中，N 为节点的个数，也就是算例的大小。因此，TSP 问题的第二种最终版模型可以表示为

$$\min \quad \sum_i \sum_j c_{ij} x_{ij} \tag{2.77}$$

$$\text{s.t.} \quad \sum_{i \in V} x_{ij} = 1, \qquad \forall j \in V, i \neq j \tag{2.78}$$

$$\sum_{j \in V} x_{ij} = 1, \qquad \forall i \in V, i \neq j \tag{2.79}$$

$$\mu_i - \mu_j + N x_{ij} \leqslant N - 1, \qquad \forall i,j \in V, i,j \neq 0, i \neq j \tag{2.80}$$

$$x_{ij} \in \{0,1\}, \mu_i \geqslant 0, \mu_i \in \mathbb{R}, \qquad \forall i,j \in V, i \neq j \tag{2.81}$$

MTZ 约束的加入，使得原问题增加了 N 个连续变量和 N^2 复杂度个逻辑约束，从代码实现上来讲是非常方便的，比起 subtour-elimination 的实现要容易得多。

接下来，在这里列几个在代码实战中可能遇到的问题。

实现这段代码时需要注意，由于 μ_i 表示访问顺序，而且 TSP 的起点和终点是一致的，如果不做处理，就会出现不可行（infeasible）的情况。为此，我们加入一个虚拟点，也就是将起始点 s 复制一份，标记为点 t，作为终止点，实际上点 s 和点 t 位置是一样的。这样处理之后，就不会存在上面介绍的不可行的情况了。另外，在代码实现的过程中，要注意在添加决策变量时判断 i 和 j 是否相等，只有当 $i \neq j$ 时，才加入决策变量。如果不做这一步判断，很可能会导致最优解出现 $x_{ii}=1, \forall i \in V$，而其余决策变量均为 0 的情形。

根据上面的注意事项，我们将模型修正一下，变成点集为 $V'=\{1,2,\cdots,N,N+1\}$，一共有 $N+1$ 个点，其中点 1 和点 $N+1$ 是同一个点，点 1 表示起点，点 $N+1$ 表示终点。模型修正为

$$\min \quad \sum_i \sum_j c_{ij} x_{ij} \tag{2.82}$$

$$\text{s.t.} \quad \sum_{i \in V} x_{ij} = 1, \qquad \forall j \in \{2,3,\cdots,N+1\}, i \neq j \tag{2.83}$$

$$\sum_{j \in V} x_{ij} = 1, \qquad \forall i \in \{1,2,\cdots,N\}, i \neq j \tag{2.84}$$

$$\mu_i - \mu_j + N x_{ij} \leqslant N-1, \qquad \forall i \in \{1,2,\cdots,N\}, j \in \{2,3,\cdots,N+1\}, i \neq j \tag{2.85}$$

$$x_{ij} \in \{0,1\}, \mu_i \geqslant 0, \mu_i \in \mathbb{R}, \qquad \forall i \in \{i,2,\cdots,N\}, j \in \{2,3,\cdots,N+1\}, i \neq j. \tag{2.86}$$

请仔细琢磨上面的约束 (2.83)、(2.84) 和 (2.85) 后面的限制条件部分的细微变化，这些都是为之后写代码打基础，以避免出现差错。

接下来，我们解释为什么 MTZ 约束有消除子环路的效果。

MTZ 这个约束为什么能够消除子环路呢？将 MTZ 约束做一个变换，得

$$\mu_i - \mu_j + 1 + N(x_{ij}-1) \leqslant 0, \quad \forall i \in \{1,2,\cdots,N\}, j \in \{2,3,\cdots,N+1\}, i \neq j$$

在上式中，$(x_{ij}-1)$ 并不是 0-1 变量，而 $(1-x_{ij})$ 才是 0-1 变量，因此该式变为

$$\mu_i - \mu_j + 1 - N(1-x_{ij}) \leqslant 0, \quad \forall i \in \{1,2,\cdots,N\}, j \in \{2,3,\cdots,N+1\}, i \neq j$$

这个约束保证了，当 $x_{ij}=1$ 时，$\mu_i - \mu_j + 1 \leqslant 0$。

任取 n $(n \leqslant N)$ 个点，它们之间被选择的总数小于或等于 $N-1$，即消除了子环路。举例来说，任取 3 个点 i,j,k，如果出现子环路，则有

$$x_{ij} = x_{jk} = x_{ki} = 1$$

$$x_{ij}+x_{jk}+x_{ki}=3$$

也就是说，根据 MTZ 约束，如果上述情况成立，则必有

$$\mu_i-\mu_j+1\leqslant 0$$
$$\mu_j-\mu_k+1\leqslant 0$$
$$\mu_k-\mu_i+1\leqslant 0$$

将以上 3 个式子相加，得

$$3\leqslant 0$$

上面不等式显然不成立，说明这个子环路不可能出现，这也就用反证法证明了，任一满足 MTZ 的点集，都不存在环路。

注意：我们的约束后的限制条件是 $\forall i\in\{1,2,\cdots,N\},j\in\{2,3,\cdots,N+1\},i\neq j$。这一点在代码实现部分是需要特意关注的。

对其他情况，任意取 n 个点，都是同样的原理。根据上述论述，容易得出，MTZ 约束可以成功地避免子环路的产生。

2.8.3 TSP 建模方法 3：1-tree 建模方法

TSP 还有一种从图论的角度建模的方式。这种建模方式，需要将原有假设加强为给定无向图 $G=(V,E)$，也就是假设一条边的往返距离是相等的。这一类的 TSP 称为对称旅行商问题（Symmetric Travelling Salesman Problem，STSP）。但是该方法需要用到一个概念：1-tree。在介绍模型之前，先来介绍这个概念。

1. 1-tree 的概念

给定一个无向图 $G=(V,E)$，$V=\{1,2,3,\cdots,n\}$ 代表图中所有点的集合，E 代表图中所有边的集合。1-tree 的定义如下。

定义 2.8.1 （Wolsey, 1998）1-tree 是在点集 $\{2,3,\cdots,n\}$ 和两条入射到点 1 的边上的一棵树，它是图 G 的一个子图。

我们来解读一下，一个 1-tree 由下面 2 部分组成（假设以点 1 为基准）：

（1）入射到点 1 的两条边；

（2）在点集 $\{2,3,\cdots,n\}$ 上的一个 tree。

图 2.20 是一个 1-tree 的几个例子。

根据上面的例子，我们可知 1-tree 具有下面的特点：

（1）有且仅有一个环；

（2）图中每个点的度 degree$\geqslant 1$；

（3）一共有 $|V|$ 条边。

2. TSP 的 1-tree 建模

在图 2.18(b) 所示的 TSP 的解中，所有点的 degree 都为 2。我们可以很容易得到，如果这个图中的边满足下面的条件，就一定是 STSP 的一个解：

$$d_i = 2, \quad \forall i \in V \tag{2.87}$$

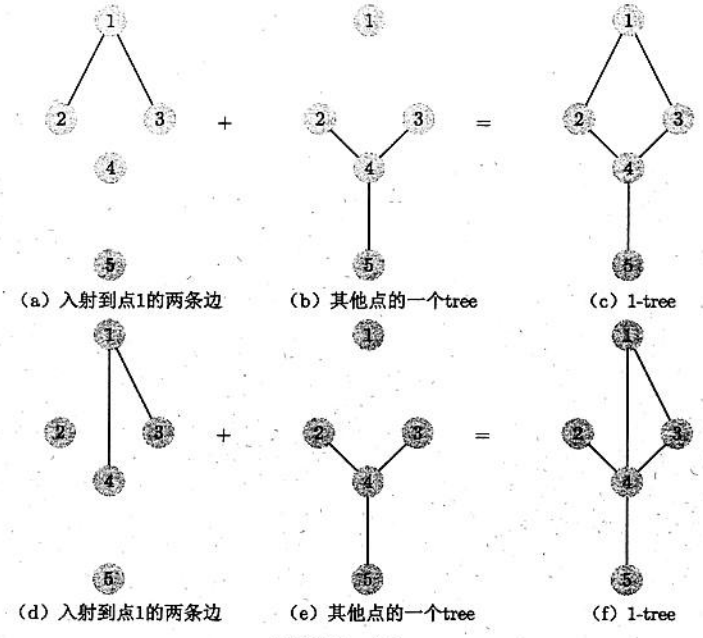


图 2.20 1-tree

而我们从此前的描述可知，1-tree 中有且仅有一个环，那么 STSP 的解就是一个特殊的 1-tree，因此我们可以利用这一点来对 STSP 进行建模。

我们利用列生成算法的思想来建模（列生成算法的理论介绍见第 13 章）。首先，我们生成一系列的 1-tree，然后我们在这些 1-tree 里面选择一个总距离最小的特殊的 1-tree（其实就是一个环），使之满足每个点的度均为 2，即 $d_i = 2, \forall i \in V$。

因此模型如下：

$$\min \quad \sum_{e \in E} c_e x_e \tag{2.88}$$

$$\text{s.t.} \quad \sum_{e \in E(i)} x_e = 2, \qquad \forall i \in V \tag{2.89}$$

$$\sum_{e \in E(S)} x_e \leqslant |S| - 1, \qquad 2 \leqslant |S| \leqslant n-1, S \subseteq V \tag{2.90}$$

$$x_e \in \{0,1\}, \qquad \forall e \in E \tag{2.91}$$

上面的模型可以再加强一些，把第一组 $|V|$ 个约束 $\sum_{e\in E(i)} x_e = 2, \forall i \in V$ 全部加起来，然后除以 2，得

$$\sum_{e\in E} x_e = |V|$$

这个约束可以加强约束模型，因此加强后的模型为

$$\min \quad \sum_{e\in E} c_e x_e \tag{2.92}$$

$$\text{s.t.} \quad \sum_{e\in E(i)} x_e = 2, \qquad \forall i \in V \tag{2.93}$$

$$\sum_{e\in E(S)} x_e \leqslant |S| - 1, \qquad 2 \leqslant |S| \leqslant n-1, S \subseteq V \tag{2.94}$$

$$\sum_{e\in E} x_e = |V| \tag{2.95}$$

$$x_e \in \{0,1\}, \qquad \forall e \in E \tag{2.96}$$

其中，$E(i)$ 表示经过点 i 的边的集合；$E(S)$ 表示起点和终点都在点集 S 中的边的集合。

在之后的章节我们还会用列生成的方法对 TSP 的 1-tree 角度建模进行进一步探讨。本节仅探讨基本的建模思路。

2.9 车辆路径规划问题

2.9.1 概述

车辆路径规划问题（Vehicle Routing Problem，VRP）是运筹学中一个非常重要的问题。该问题由单纯形法的发明人，著名的 G. B. Dantzig 于 1959 年提出（Dantzig and Ramser, 1959），几十年来一直是运筹优化领域一个经久不衰的热点问题。其定义如下。

给定一个有向图 $G = (V, A)$，其中 $V = \{0, 1, 2, \cdots\}$ 为图中的节点，其中 0 点表示仓库，C 为客户点的集合，且 $V = C \cup \{0\}$。A 为图中弧的集合。给定一个车的集合 K，每辆车的最大容量为 Q，在每个节点 $i \in V$ 处，都对应一个需求 q_i，且对于仓库点 $q_0 = 0$，其余点需求均为正。VRP 的目标就是设计一系列路径，使得每辆车均从起始点出发，满足所有客户点的需求，并且最终返回起始点。每个客户点能且仅能被一辆车访问一次。

图 2.21 为一个简单的 VRP 的例子。

VRP 发展至今，已经有非常多的变体。最基本的 VRP 叫作带容量的车辆路径规划问题（Capacitated Vehicle Routing Problem，CVRP）。在 CVRP 中，需要考虑每辆车的容量约束、车辆的路径约束和装载量的约束。而为了考虑配送的时间要求，带时间窗的车辆路径规划问题（Vehicle Routing Problem with Time Windows，VRPTW）应运而生。VRPTW 不仅考虑 CVRP 的所有约束，而且还考虑了时间窗约束。也就是每个顾客点都

对应一个时间窗 $[e_i, l_i]$，其中 e_i 和 l_i 分别代表该点的最早到达时间和最晚到达时间。顾客点 $i \in V$ 的需求必须要在其时间窗内被送达。

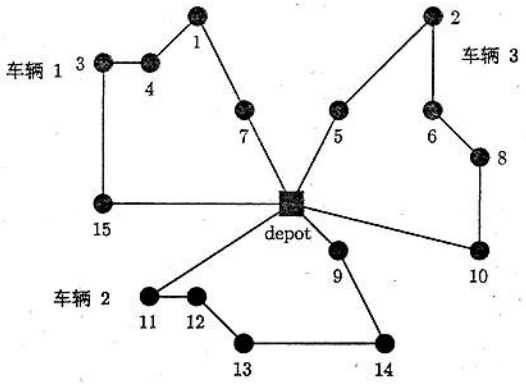


图 2.21 一个简单的 VRP 的例子

当然，在实际物流配送中，还有很多其他因素，也因此引申出了很多 VRP 的其他变体，例如：

（1）带取货和送货时间窗的 CVRP；

（2）多车场 VRP；

（3）带时间窗约束的多车场 VRP；

（4）周期性 VRP；

（5）带时间窗的周期性 VRP；

（6）考虑存取货的 VRP。

本节中，我们主要探讨考虑时间窗的车辆路径规划问题（VRPTW）。关于 VRPTW 的资源非常多，这里介绍一个质量比较高的工具，就是 Google 公司开发的 OR Tools，参见网址 `https://developers.google.com/optimization/routing/cvrp`，该网站可以很方便地查看各种 VRP 的介绍和相应的基础代码。另外，关于 VRP 问题的各种变种问题的模型、文献综述以及理论的详细介绍，参见教材 *Vehicle Routing: Problems, Methods, and Applications*（Toth and Vigo, 2014）。

此外，也有很多 VRP 相关的标杆算例资源，比如从事 VRP 相关研究的 Marius M. Solomon 在 1987 年发表在 *Operations Research* 上的文章中采用的算例就是一个非常经典的算例集合（Solomon and Marius, 1987）。关于该 VRP 标杆算例（benchmark）的下载网址和已知最优解等信息，可以去网址 `https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/100-customers/`进行查看，如图 2.22 所示。

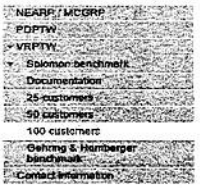
2.9.2 VRPTW 的一般模型

VRPTW 可以建模为一个混合整数规划（MIP）。在给出完整数学模型之前，我们首先引入下面的决策变量：

$$x_{ijk}=\begin{cases}1, & \text{如果在最优解中，弧 } (i,j) \text{ 被车辆 } k \text{ 选中}\\ 0, & \text{其他}\end{cases}$$

$s_{ik}=$ 车辆 k 到达 i 的时间

模型中涉及的其他参数为：t_{ij} 表示车辆在弧 (i,j) 上的行驶时间，M 为一个足够大的正数。



100 customers

Here you find instance definitions and best known solutions for the 100 customer instances of Solomon's VRPTW benchmark problems from 1987. The version reported here has a hierarchical objective: 1) Minimize number of vehicles 2) Minimize total distance. Distance and time should be calculated with double precision, total distance results are rounded to two decimals. Exact methods typically use a monolithic total distance objective and use integral or low precision distance and time calculations. Hence, results are not directly comparable. By clicking on an instance name in blue, you will open a file with a specification of the detailed solution.

Instance definitions (text)

See Solomon's web page http://web.cba.neu.edu/~msolomon/problems.htm .

As a backup, you will find a zip-file with the 100 instance definitions here.

Best known solution values

See Solomon's web page http://web.cba.neu.edu/~msolomon/problems.htm .

A few new results have been added in the updated table below.

Instance	Vehicles	Distance	Reference	Comment
c101	10	828.94	RT	Detailed solution by SAM::OPT
c102	10	828.94	RT	Detailed solution by SAM::OPT
c103	10	828.06	RT	Detailed solution by SAM::OPT
c104	10	824.78	RT	Detailed solution by SAM::OPT
c105	10	828.94	RT	Detailed solution by SAM::OPT
c106	10	828.94	RT	Detailed solution by SAM::OPT
c107	10	828.94	RT	Detailed solution by SAM::OPT
c108	10	828.94	RT	Detailed solution by SAM::OPT
c109	10	828.94	RT	Detailed solution by SAM::OPT
c201	3	591.56	RT	Detailed solution by SAM::OPT
c202	3	591.56	RT	Detailed solution by SAM::OPT
c203	3	591.17	RT	Detailed solution by SAM::OPT
c204	3	590.60	RT	Detailed solution by SAM::OPT
c205	3	588.88	RT	Detailed solution by SAM::OPT
c206	3	588.49	RT	Detailed solution by SAM::OPT
c207	3	588.29	RT	Detailed solution by SAM::OPT
c208	3	588.32	RT	Detailed solution by SAM::OPT
r101	19	1650.80	H	See note below
r102	17	1486.12	RT	Detailed solution by SAM::OPT
r103	13	1292.68	LLH	Detailed solution by Shobb
r104	9	1007.31	MBD	See note below
r105	14	1377.11	RT	Detailed solution by SAM::OPT
r106	12	1252.03	MBD	See note below
r107	10	1104.66	S97	Detailed solution by Shobb
r108	9	960.88	BBB	Detailed solution from BVH
r109	11	1194.73	HG	Detailed solution by Shobb
r110	10	1118.84	MBD	See note below
r111	10	1096.72	RGP	Detailed solution by Shobb
r112	9	982.14	GTA	

图 2.22 Solomon VRP 标杆算例最优解信息（部分）

注：参见 https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/100-customers/.

我们参考文献（Desaulniers et al., 2006）给出 VRPTW 的标准模型如下：

$$\min \quad \sum_{k\in K}\sum_{i\in V}\sum_{j\in V} c_{ij}x_{ijk} \tag{2.97}$$

$$\text{s.t.} \quad \sum_{k\in K}\sum_{j\in V} x_{ijk}=1, \qquad \forall i\in C \tag{2.98}$$

$$\sum_{j\in V} x_{0jk}=1, \qquad \forall k\in K \tag{2.99}$$

$$\sum_{i\in V} x_{ihk} - \sum_{j\in V} x_{hjk} = 0, \qquad \forall h \in C, \forall k \in K \tag{2.100}$$

$$\sum_{i\in V} x_{i,n+1,k} = 1, \qquad \forall k \in K \tag{2.101}$$

$$\sum_{i\in C} q_i \sum_{j\in V} x_{ijk} \leqslant Q, \qquad \forall k \in K \tag{2.102}$$

$$s_{ik} + t_{ij} - M(1 - x_{ijk}) \leqslant s_{jk}, \qquad \forall (i,j) \in A, \forall k \in K \tag{2.103}$$

$$e_i \leqslant s_{ik} \leqslant l_i, \qquad \forall i \in V, \forall k \in K \tag{2.104}$$

$$x_{ijk} \in \{0,1\}, \qquad \forall (i,j) \in A, \forall k \in K \tag{2.105}$$

其中，目标函数 (2.97) 为最小化所有车辆的总行驶距离；约束 (2.98)~(2.101) 保证了每辆车必须从仓库出发，经过一个点就离开那个点，最终返回到仓库。约束 (2.102) 为车辆的容量约束。约束 (2.103) 和 (2.104) 是时间窗约束，保证了车辆到达每个顾客点的时间均在时间窗内，点 $n+1$ 是点 o 的一个备份，是为了方便实现。

关于 M 的取值，实际上可以取非常大的正数。但是为了提高求解效率，拉紧约束，我们采用下面的取值方法（Desaulniers et al., 2006）：

$$M = \max\{b_i + t_{ij} - a_j\}, \quad \forall (i,j) \in A \tag{2.106}$$

VRPTW 已经被证明是 NP-hard 问题，其求解复杂度随着问题规模的增加急剧增加，求解较为困难。目前为止，求解 VRPTW 比较高效的精确算法是分支定价算法以及分支定价切割算法。第 15 章有对分支定价算法的详细讲解。

第3章　整数规划建模技巧

整数规划的建模是非常有技巧性的。基础的关系，如和、差等，比较容易建模。但是一些复杂的关系，如互斥关系、并列关系等，就需要借助一些建模技巧才能完成。除此之外，还有一些非线性模型，也可以通过线性化的手段将其等价转化为线性模型，从而更高效地求解。建立正确、紧凑的模型是解决运筹优化问题非常重要和基础的部分，也是该领域从业者必须具备的能力。本章主要介绍逻辑约束和线性化两大部分。其中逻辑约束部分可以帮助读者准确地用数学语言刻画出实际问题，线性化部分可以用于模型重构和转化。

3.1 逻辑约束

在使用运筹学建模过程中，我们经常会遇到逻辑约束问题。那到底什么是逻辑约束呢？我们以投资组合为例进行说明。假如某商人考虑从若干种投资组合中选择投资，他目前对于每一项的投资的净收益和购买价格是已知的。在进行投资决策时，除了要满足总购买价格小于可用于投资的总资产外，该商人还要考虑一些约束，如：最多可以投资两项；如果选择投资 A，那么就必须选择投资 B；如果选择投资 C，那么就无法再选择投资 D；等等。这些约束就属于逻辑约束。对逻辑约束进行数学建模，通常需要灵活使用如下两个命题和两个条件。

3.1.1 两个命题

在对逻辑约束条件建模时，通常需要使用如下两个命题。

（1）如果 0-1 型变量 $y=0$, 那么变量 $x \leqslant 0$（x 不一定是 0-1 型变量，还可以是一个线性表达式）。

我们通常把该关系约束表示成：$x - My \leqslant 0$，其中 M 是变量 x 的一个上界。

（2）逆否命题。

如果满足约束 A，那么就需要满足约束 B。 $\Longleftrightarrow$ 如果不满足约束 B，那么就不满足约束 A。

3.1.2 二选一约束条件

在数学规划问题中，经常会出现如下情况：

$$f(x_1, x_2, \cdots, x_n) \leqslant 0 \tag{3.1}$$

$$g(x_1, x_2, \cdots, x_n) \leqslant 0 \tag{3.2}$$

如果我们希望保证至少满足 (3.1) 和 (3.2) 中的一个约束条件，则它被称作二选一约束条件（Either-Or）。

对于这种逻辑约束条件，通常引入 0-1 型变量。此逻辑约束条件可表示如下：

$$f(x_1, x_2, \cdots, x_n) \leqslant My \tag{3.3}$$

$$g(x_1, x_2, \cdots, x_n) \leqslant M(1-y) \tag{3.4}$$

在 (3.3) 和 (3.4) 中，y 是 0-1 型变量，M 是一个足够大的数。这两条约束可以至少满足 (3.1) 和 (3.2) 中的一个约束条件。

我们可以这样理解，如果 $y=0$，那么 (3.3) 和 (3.4) 将变成 $f \leqslant 0$ 和 $g \leqslant M$，必定满足 (3.1)，也许还满足 (3.2)。类似地，如果 $y=1$，那么 (3.3) 和 (3.4) 将变成 $f \leqslant M$ 和 $g \leqslant 0$，必定满足 (3.2)，也许还满足 (3.1)。因此，无论 $y=0$ 还是 $y=1$，(3.3) 和 (3.4) 都将保证至少满足 (3.1) 和 (3.2) 中的一个约束条件。

在 Gurobi 中，这类约束可以通过构建 And 和 Or 约束实现。相应的函数为 addGenConstrAnd (resvar, vars, name="") 和 addGenConstrOr(resvar, vars, name="")。

首先来看 And 约束。假设有一系列 0-1 变量 $y, x_1, x_2, \cdots, x_n$，考虑下面的关系。

$$y = \text{and}\{x_1, x_2, \cdots, x_n\}$$

式中，当且仅当所有 $x_1, x_2, \cdots, x_n$ 的取值均为 1 时，$y=1$，否则 $y=0$。如果 $x_1, x_2, \cdots, x_n$ 为一般变量（即连续型变量），上面的约束依然可以运行成功，但是 Gurobi 会在内部自动将参与 And 约束的变量强制转换成 0-1 变量。下面为代码示例。

And constraints

```
# define variables
x_1 = model.addVar(vtype = GRB.BINARY)
x_2 = model.addVar(vtype = GRB.BINARY)
x_3 = model.addVar(vtype = GRB.BINARY)
y = model.addVar(vtype = GRB.BINARY)
# 也可以将其定义为连续变量
# x_1 = model.addVar(lb = -GRB.INFINITY, ub = GRB.INFINITY, vtype = GRB.
      # CONTINUOUS)
# x_2 = model.addVar(lb = -GRB.INFINITY, ub = GRB.INFINITY, vtype = GRB.
      # CONTINUOUS)
# x_3 = model.addVar(lb = -GRB.INFINITY, ub = GRB.INFINITY, vtype = GRB.
      # CONTINUOUS)
# y = model.addVar(lb = 0, ub = GRB.INFINITY, vtype = GRB.CONTINUOUS)

# y = and(x_1, x_2, x_3)
model.addGenConstrAnd(y, [x_1, x_2, x_3], "andconstr")
# overloaded forms
```

```
model.addConstr(y == and_([x_1, x_2, x_3]), "andconstr")
model.addConstr(y == and_(x_1, x_2, x_3), "andconstr")
```

然后来看 Or 约束。假设有一系列 0-1 变量 $y, x_1, x_2, \cdots, x_n$，考虑下面的关系。

$$y = \text{or}\{x_1, x_2, \cdots, x_n\}$$

式中，当且仅当存在 $x_i, i = 1, 2, \cdots, n$ 的取值为 1 时，$y = 1$；当 $x_1, x_2, \cdots, x_n$ 的取值均为 0 时，$y = 0$。下面为代码示例。

And constraints

```
# define variables
x_1 = model.addVar(vtype = GRB.BINARY)
x_2 = model.addVar(vtype = GRB.BINARY)
x_3 = model.addVar(vtype = GRB.BINARY)
y = model.addVar(vtype = GRB.BINARY)

# y = or(x_1, x_2, x_3)
model.addGenConstrOr(y, [x_1, x_2, x_3], "orconstr")
# overloaded forms
model.addConstr(y == or_([x_1, x_2, x_3]), "orconstr")
model.addConstr(y == or_(x_1, x_2, x_3), "orconstr")
```

同样地，参与 Or 约束的所有决策变量将被自动强制转换成 0-1 变量。

3.1.3 指示约束条件

在很多情况下，我们希望保证，如果满足约束条件 $f(x_1, x_2, \cdots, x_n) > 0$，那么必须满足约束条件 $g(x_1, x_2, \cdots, x_n) \geqslant 0$；如果没有满足 $f(x_1, x_2, \cdots, x_n) > 0$，那么可以满足也可以不满足约束条件 $g(x_1, x_2, \cdots, x_n) \geqslant 0$。

为了保证这个约束，我们引入 0-1 型变量 y，加入如下约束条件：

$$-g(x_1, x_2, \cdots, x_n) \leqslant My \tag{3.5}$$

$$f(x_1, x_2, \cdots, x_n) \leqslant M(1 - y) \tag{3.6}$$

通常，M 是一个足够大的正数，使得 $f \leqslant M$ 和 $-g \leqslant M$ 对于满足问题中其他约束条件中的 $x_1, x_2, \cdots, x_n$ 所有值都成立。

可以看到，如果 $f > 0$，那么只有当 $y = 0$ 时才满足 (3.6)，当 $y = 0$ 时，由 (3.5) 可知，$g \geqslant 0$，这是我们需要满足的逻辑约束条件。此外，如果没有满足 $f > 0$，那么 (3.6) 允许 $y = 0$ 或 $y = 1$，$g < 0$ 和 $g \geqslant 0$ 的情况都可能出现。而选择 $y = 1$ 将满足 (3.5)。

指示约束条件在 Gurobi 中可以通过调用函数 addGenConstrIndicator(binvar, binval, lhs, sense=None, rhs=None, name="") 实现。该函数各个参数的含义如下。

- binvar：0-1 指示变量对象；
- binval：0-1 指示变量的取值（True 或者 False）；
- lhs (float, Var, LinExpr, or TempConstr)：与 0-1 指示变量相关联的线性约束的左端项表达式；
- sense (char)：线性约束的符号，可选值为 GRB.LESS_EQUAL，GRB.EQUAL，GRB.GREATER_EQUAL；
- rhs (float)：线性约束的右端项；
- name (string, optional)：约束的名称。

例如，我们想实现如果 $y=1$，则 $x_1+2x_2+x_3=1$，此类约束可以通过下面的代码实现。

Indicator constraints

```
# y = 1 → x_1 + 2 x_2 + x_3 = 1
model.addGenConstrIndicator(y, True, x_1 + 2*x_2 + x_3, GRB.EQUAL, 1.0)
# alternative form
model.addGenConstrIndicator(y, True, x_1 + 2*x_2 + x_3 == 1.0)
# overloaded form
model.addConstr((y == 1) >> (x_1 + 2*x_2 + x_3 == 1.0))
```

3.2 线 性 化

3.2.1 分段线性函数线性化

下面将介绍如何使用 0-1 型变量建立涉及分段线性函数的最优化问题模型。

分段线性函数由几条直线段组成，分段线性函数斜率发生变化（或者函数的定义范围结束）的点称作函数的间断点。

如图 3.1 所示，该分段线性函数由 4 条直线段组成，横坐标 x 为 0、10、30、40、50 的点都是这个函数的间断点（Winston，2004）。

分段线性函数不是线性函数，一般不能使用线性规划直接对涉及线性分段函数的最优化问题进行求解。但是我们可以利用 0-1 型变量，把分段线性函数表示成线性形式。

假设一个分段线性函数 $f(x)$ 在 x 为 $b_1,b_2,\cdots,b_n$ 处有分段点。对于某个 $k(k=1,2,\cdots,n-1)$，有 $b_k \leqslant x \leqslant b_{k+1}$。因此对于某个数 $z_k(0 \leqslant z_k \leqslant 1)$，可以把 x 记作

$$x = z_k b_k + (1-z_k)b_{k+1} \tag{3.7}$$

由于 $f(x)$ 对于 $b_k \leqslant x \leqslant b_{k+1}$ 是线性的，所以我们可以把 $f(x)$ 记作

$$f(x) = z_k f(b_k) + (1-z_k)f(b_{k+1}) \tag{3.8}$$

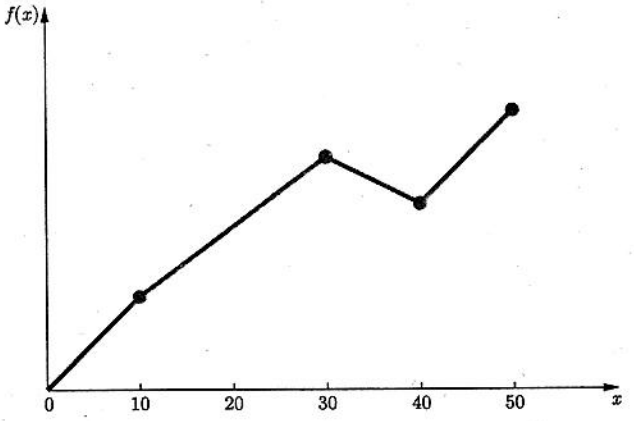


图 3.1 分段线性函数

接下来介绍利用线性约束条件和 0-1 型变量表示分段线性函数的方法。

（1）第 1 步：在最优化问题中出现 $f(x)$ 的地方，用 $z_1 f(b_1) + z_2 f(b_2) + \cdots + z_n f(b_n)$ 代替 $f(x)$。

（2）第 2 步：在问题中添加如下约束条件：

$$
\begin{aligned}
& z_1 \leqslant y_1 \\
& z_2 \leqslant y_1 + y_2 \\
& z_3 \leqslant y_2 + y_3 \\
& \vdots \\
& z_{n-1} \leqslant y_{n-2} + y_{n-1} \\
& z_n \leqslant y_{n-1} \\
& y_1 + y_2 + \cdots + y_{n-1} = 1 \\
& z_1 + z_2 + \cdots + z_n = 1 \\
& x = z_1 b_1 + z_2 b_2 + \cdots + z_n b_n \\
& y_i \in \{0,1\}, \qquad \forall i = 1,2,\cdots,n-1 \\
& z_i \geqslant 0, \qquad \forall i = 1,2,\cdots,n
\end{aligned}
$$

针对分段函数线性化，我们还有另一种形式的描述。

假如有如图 3.2 所示的分段线性函数。

该分段线性函数 x 轴和 y 轴的分段区间为

$$
\begin{aligned}
& x \text{ 轴：} [0,a_1),[a_1,a_2),[a_2,a_3] \\
& y \text{ 轴：} [0,b_1),[b_1,b_2),[b_2,b_3]
\end{aligned}
$$

间断点则为

$$(0,0),(a_1,b_1),(a_2,b_2),(a_3,b_3)$$

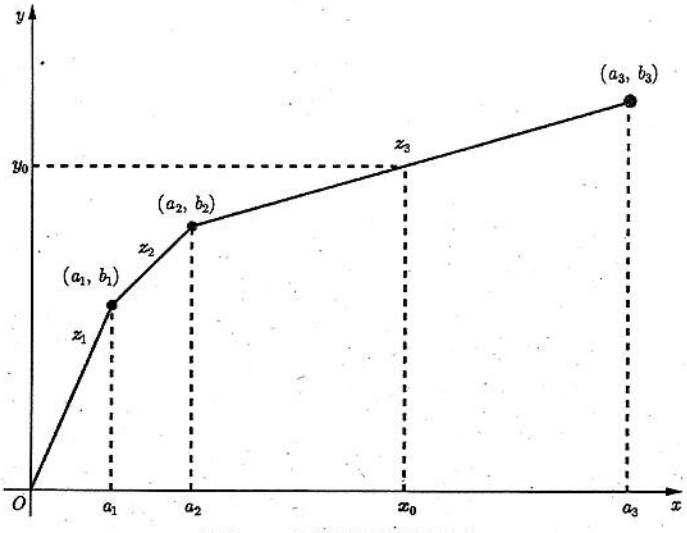


图 3.2 分段线性函数线性化

这里的目的是给定一个 x_0，通过约束，使其获得相应的 y_0。为此，引入一个辅助变量 $z \in \{0,1\}$，该变量的取值与 x 所在的横轴区间有关，即

$$z_i = \begin{cases} 1, & x \text{ 落在第 } i \text{ 个分段区间内} \\ 0, & \text{其他} \end{cases}$$

下面不加证明地给出一个结论。

点 $X^{(1)}(x_1,y_1)$ 和 $X^{(2)}(x_2,y_2)$ 为平面内两点，则 $X^{(1)}$ 和 $X^{(2)}$ 连线上任意一点 X 可以表示为

$$X = (1-\alpha)X^{(1)} + \alpha X^{(2)} \tag{3.9}$$

且 $X^{(1)}X = \alpha X^{(1)}X^{(2)}$。

这一点很容易得到，因为只看横坐标，当 $X^{(1)}X = \alpha X^{(1)}X^{(2)}$ 时，有

$$x = x_1 + \alpha(x_2 - x_1) = (1-\alpha)x_1 + \alpha x_2$$

因此，对于曲线上任何一个点 (x,y)，均可被表示为如下的形式：

$$x = \beta_0 a_0 + \beta_1 a_1 + \beta_2 a_2 + \beta_3 a_3 = \sum_{k \in K} \beta_k a_k \tag{3.10}$$

$$y=\beta_0 b_0+\beta_1 b_1+\beta_2 b_2+\beta_3 b_3=\sum_{k\in K}\beta_k b_k \tag{3.11}$$

并且，$\beta_0,\beta_1,\beta_2,\beta_3$ 中只能有 1 个或者 2 个大于 0，其余均等于 0，并且满足

$$\beta_0+\beta_1+\beta_2+\beta_3=1 \tag{3.12}$$

下面来看如何将分段线性函数转化成约束（这里假设 $a_0=0$，但是 β_0 是不能忽略的，必须有，也就是 β 比 z 多 1 个），假设一共有 $K(K\geqslant 1)$ 个区间段。

(1) 对于第 1 个区间段，因为都是左闭右开区间，如果 $z_1=1$，则必然有 $\beta_0>0,\beta_1\geqslant 0$。因为是左闭右开区间，左边一定能取到，也有可能取到左边区间端点。但是右边断点一定取不到，因此 $\beta_1<1$，故有 $\beta_0>0$，并且 β_0 也可能取到 1，取到 1 时，说明刚好落在区间左端点。如果 $z_1=0$，则 $\beta_0\geqslant 0,\beta_1\geqslant 0$。因此，$z$ 的取值在这个层面没有必然联系。

(2) 对于前 $K-1$ 个区间，均与上述同理。

(3) 先看第一段，$z_1=0$，必有 $\beta_0=0,\beta_1\geqslant 0\Rightarrow \beta_0-Mz_1\leqslant 0$。

(4) 再看最后一段，如果 $z_K=0$，必有 $\beta_K=0,\beta_{K-1}\geqslant 0\Rightarrow \beta_K-Mz_K\leqslant 0$。

(5) 最后看中间的区间段，当 $z_{k-1}=0$ 且 $z_k=0$ 时，必有 $\beta_{k-1}=0\Rightarrow \beta_{k-1}-M(z_{k-1}+z_k)\leqslant 0$。

由于 M 是 z 的一个上界，这里取 $M=1$，因此只需要添加如下的约束：

$$x=\sum_{k=0}^{K}\beta_k a_k \tag{3.13}$$

$$y=\sum_{k=0}^{K}\beta_k b_k \tag{3.14}$$

$$\sum_{k=0}^{K}\beta_k=1 \tag{3.15}$$

$$\sum_{k=0}^{K}z_k=1 \tag{3.16}$$

$$\beta_0-z_1\leqslant 0 \tag{3.17}$$

$$\beta_K-z_K\leqslant 0 \tag{3.18}$$

$$\beta_{k-1}-(z_{k-1}+z_k)\leqslant 0,\quad \forall k\in K,k\neq 0,k\neq K \tag{3.19}$$

$$z_0=0 \tag{3.20}$$

$$0\leqslant \beta_k\leqslant 1,z_k\in\{0,1\},\quad \forall k\in K \tag{3.21}$$

此外，对于 $\ln x,\log x,\sin x,\cos x,\tan x,\mathrm{e}^x,a^x,x^a$ 等形式的非线性函数，均可通过分段线性近似，进而用分段函数线性化的方法将其线性化。Gurobi 中也提供了相应的函数，可以很方便地实现这些非线性函数的分段函数线性近似，具体介绍见第 7 章。

在 Gurobi 中，分段函数的线性化是可以自动完成的，不用像上面描述的那样复杂。如果是 Python 调用 Gurobi，则对应的函数为 addGenConstrPWL(xvar, yvar, xpts, ypts, name="")。其中各个参数的意义如下。

- xvar：决策变量 x；
- yvar：决策变量 y；
- xpts (list of float)：分段函数 $y=f(x)$ 的 x 轴分段点；
- ypts (list of float)：分段函数 $y=f(x)$ 的 y 轴分段点；
- name (string, optional)：该约束的名称。

接下来展示一个具体例子。

Piecewise-linear function

```
# define Piecewise-linear constraints
PWL_cons = model.addGenConstrPWL(x, y, [0, 2, 3], [1, 4, 3], "PWLConstr")
```

上文中提到的 $\ln x, \log x, \sin x, \cos x, \tan x, \mathrm{e}^x, a^x, x^a$ 等形式的非线性函数，虽然不能直接调用 addGenConstrPWL() 函数完成分段函数线性化，但是可以通过调用下面的函数间接完成线性化。

- addGenConstrExp(xvar, yvar, name="", options="")：$y=\mathrm{e}^x$；
- addGenConstrExpA(xvar, yvar, a, name="", options="")：$y=a^x$；
- addGenConstrLog(xvar, yvar, name="", options="")：$y=\ln x$；
- addGenConstrLogA(xvar, yvar, a, name="", options="")：$y=\log_a x$；
- addGenConstrPow(xvar, yvar, a, name="", options="")：$y=x^a$；
- addGenConstrSin(xvar, yvar, name="", options="")：$y=\sin(x)$；
- addGenConstrCos(xvar, yvar, name="", options="")：$y=\cos(x)$；
- addGenConstrTan(xvar, yvar, name="", options="")：$y=\tan(x)$。

3.2.2 含绝对值形式的线性化

对于目标函数含有绝对值的情况，如 $\min|x|$，可以通过引入辅助变量进行线性化。考虑下面的含有绝对值的非线性规划。

$$\begin{aligned}\min \quad & |x_1|+|x_2| \\ & x_1, x_2 \in \mathbb{R}\end{aligned}$$

首先我们引入辅助变量，令

$$x_1^+ = \max\{0, x_1\}, \quad x_1^- = \max\{0, -x_1\}$$

$$x_2^+ = \max\{0, x_2\}, \quad x_2^- = \max\{0, -x_2\}$$

可以得出：

$$|x_1| = x_1^+ + x_1^-, \quad x_1 = x_1^+ - x_1^-$$
$$|x_2| = x_2^+ + x_2^-, \quad x_2 = x_2^+ - x_2^-$$

因此，原问题等价于：

$$\begin{aligned} \min \quad & x_1^+ + x_1^- + x_2^+ + x_2^- \\ & x_1 = x_1^+ - x_1^- \\ & x_2 = x_2^+ - x_2^- \\ & x_1, x_2 \in \mathbb{R}, x_1^+, x_1^-, x_2^+, x_2^- \geqslant 0 \end{aligned}$$

这就完成了线性化。

在 Gurobi 中，可以通过调用函数 addGenConstrAbs(resvar, argvar, name="") 或者函数 abs_() 很容易实现绝对值的线性化。比如，我们要实现上述模型的线性化，只需要编写下面的代码即可。

Abs constraints

```
model = Model()

x_1 = model.addVar(lb = -GRB.INFINITY, ub = GRB.INFINITY, vtype = GRB.
      CONTINUOUS)
x_2 = model.addVar(lb = -GRB.INFINITY, ub = GRB.INFINITY, vtype = GRB.
      CONTINUOUS)
y_1 = model.addVar(lb = 0, ub = GRB.INFINITY, vtype = GRB.CONTINUOUS)
y_2 = model.addVar(lb = 0, ub = GRB.INFINITY, vtype = GRB.CONTINUOUS)

# method 1
model.addGenConstrAbs(y_1, x_1)  # y_1 = abs(x_1)
model.addGenConstrAbs(y_2, x_2)  # y_2 = abs(x_2)

# method 2 : 使用重载过的函数abs_()   (overloaded form)
# model.addConstr(y_1 == abs_(x_1) )  # y_1 = abs(x_1)
# model.addConstr(y_2 == abs_(x_2) )  # y_2 = abs(x_2)

model.setObjective(y_1 + y_2, GRB.MINIMIZE)  # min abs(x_1) + abs(x_2)
```

上述例子中，我们通过引入两个非负的辅助变量 y_1, y_2，并且令 $y_1 = |x_1|, y_2 = |x_2|$，然后将目标函数设置为 min $y_1 + y_2$，从而实现了线性化。

之后的内容部分参考自 Gurobi 的教程和用户手册。

3.2.3 含乘积形式的线性化

若目标函数含有两个变量乘积形式，则可以通过引入变量 y 进行线性化。由于变量有可能是 0-1 变量或者整数变量，所以分如下 4 种情况进行讨论。

第 1 种情况：

$$\begin{aligned} \min \quad & x_1 x_2 \\ \text{s.t.} \quad & x_1 \in \{0,1\} \\ & x_2 \in \{0,1\} \end{aligned}$$

令 $y = x_1 x_2$，则上述非线性项可以通过添加下面的约束实现线性化：

$$\begin{aligned} \min \quad & y \\ \text{s.t.} \quad & y \leqslant x_1 \\ & y \leqslant x_2 \\ & y \geqslant x_1 + x_2 - 1 \\ & x_1, x_2, y \in \{0,1\} \end{aligned}$$

第 2 种情况：

$$\begin{aligned} \min \quad & x_1 x_2 \\ \text{s.t.} \quad & x_1 \in \{0,1\} \\ & x_2 \in [0,u] \end{aligned}$$

同样地，令 $y = x_1 x_2$，则上述非线性项可以通过添加下面的约束实现线性化：

$$\begin{aligned} \min \quad & y \\ \text{s.t.} \quad & y \leqslant u x_1 \\ & y \leqslant x_2 \\ & y \geqslant x_2 - u(1 - x_1) \\ & x_1 \in \{0,1\}, x_2, y \in [0,u] \end{aligned}$$

第 3 种情况：

$$\begin{aligned} \min \quad & x_1 x_2 \\ \text{s.t.} \quad & x_1 \in \{0,1\} \\ & x_2 \in [l,u] \end{aligned}$$

令 $y = x_1 x_2$，则上述非线性项可以通过添加下面的约束实现线性化：

$$\min \quad y$$

$$
\begin{aligned}
\text{s.t.}\quad & y \leqslant x_2 \\
& y \geqslant x_2 - u(1-x_1) \\
& lx_1 \leqslant y \leqslant ux_1 \\
& x_1 \in \{0,1\} \\
& x_2 \in [l,u] \\
& y \in [0,u]
\end{aligned}
$$

第 4 种情况：

当 x_1 和 x_2 都是连续变量时，是不能等价线性化的，只能给出一个较紧的近似。对于具体细节，读者可以参考文献（Sherali and Alameddine, 1992）。

3.2.4 含分式形式的线性化

本部分参考自 gurobi 的教学文档。若目标函数是如下分式形式：

$$
\begin{aligned}
\min\quad & \frac{\sum_i (c_i x_i + \alpha)}{\sum_i (d_i x_i + \beta)} \\
\text{s.t.}\quad & \sum_i a_{ij} x_i \leqslant b_j, \qquad \forall j \in J \\
& \sum_i d_i x_i + \beta > 0 \\
& x_i \geqslant 0, \qquad \forall i \in I
\end{aligned}
\tag{3.22}
$$

可以通过引入如下变量 y 进行线性化。令

$$
y = \frac{1}{\sum_i (d_i x_i + \beta)} > 0
$$

代入式 (3.22) 得

$$
\begin{aligned}
\min\quad & \sum_i (c_i x_i y + \alpha y) \\
\text{s.t.}\quad & \sum_i a_{ij} x_i \leqslant b_j, \qquad \forall j \in J \\
& \sum_i d_i x_i y + \beta y = 1 \\
& y > 0, x_i \geqslant 0, \qquad \forall i \in I
\end{aligned}
$$

令

$$
z_i = x_i y
$$

得线性化最终形式为

$$\begin{aligned}
\min \quad & \sum_i (c_i z_i + \alpha y) \\
\text{s.t.} \quad & \sum_i a_{ij} z_i \leqslant b_j y, \qquad \forall j \in J \\
& \sum_i d_i z_i + \beta y = 1 \\
& y > 0, z_i \geqslant 0, \qquad \forall i \in I
\end{aligned}$$

3.2.5 含 max/min 形式的线性化

对于含有 max 形式的约束，如下式所示：

$$z = \max\{x, y, 3\}$$

可以通过如下方式进行线性化 (其中，M 为一个足够大的正数，下同)

$$\begin{aligned}
x \leqslant z, y \leqslant z, 3 \leqslant z \\
x \geqslant z - M(1 - u_1) \\
y \geqslant z - M(1 - u_2) \\
3 \geqslant z - M(1 - u_3) \\
u_1 + u_2 + u_3 \geqslant 1 \\
u_1, u_2, u_3 \in \{0, 1\}
\end{aligned}$$

对于含有 min 形式的约束，如下式所示：

$$z = \min\{x, y, 3\}$$

可以通过如下方式进行线性化：

$$\begin{aligned}
x \geqslant z, y \geqslant z, 3 \geqslant z \\
x \leqslant z + M(1 - u_1) \\
y \leqslant z + M(1 - u_2) \\
3 \leqslant z + M(1 - u_3) \\
u_1 + u_2 + u_3 \geqslant 1 \\
u_1, u_2, u_3 \in \{0, 1\}
\end{aligned}$$

对于含有 max 和 min 的情形，在 Gurobi 中可以通过调用函数 `addGenConstrMax(resvar, vars, constant=None, name="")` 和 `addGenConstrMin(resvar, vars, constant=None, name="")` 实现。

例如，我们考虑下面的表达式：

$$y = \max\{x_1, x_2, \cdots, x_n, c\}$$

其中，$x_1, x_2, \cdots, x_n$ 为决策变量，c 为常数。则相应的代码如下：

Max constraints

```
# y = max(x_1, x_2, x_3, 10.0)
model.addGenConstrMax(y, [x_1, x_2, x_3], 10.0, "maxconstr")

# alternative form
model.addGenConstrMax(y, [x_1, x_2, x_3, 10.0], name="maxconstr")

# overloaded forms
model.addConstr(y == max_([x_1, x_2, x_3, 10.0]), name="maxconstr")
model.addConstr(y == max_(x_1, x_2, x_3, 10.0), name="maxconstr")
```

对于 min 的情形，处理方法类似。考虑下面的表达式。

$$y = \min\{x_1, x_2, \cdots, x_n, c\}$$

其中，$x_1, x_2, \cdots, x_n$ 为决策变量，c 为常数。相应的代码如下：

Min constraints

```
# y = min(x_1, x_2, x_3, 10.0)
model.addGenConstrMin(y, [x_1, x_2, x_3], 10.0, "minconstr")

# alternative form
model.addGenConstrMin(y, [x_1, x_2, x_3, 10.0], name="minconstr")

# overloaded forms
model.addConstr(y == min_([x_1, x_2, x_3, 10.0]), name="minconstr")
model.addConstr(y == min_(x_1, x_2, x_3, 10.0), name="minconstr")
```

第4章　大规模线性规划的对偶

4.1　对偶理论概述

对偶理论（Duality Theory）在运筹学数学规划部分占据着举足轻重的地位，也属于比较高阶的理论。

本书着重讲解对偶理论的一些应用，以及如何写出一些大规模线性规划的对偶。对于对偶理论的非常深层次的内容，我们暂时不涉及。首先我们来介绍一下掌握对偶理论的重要性。在实际的研究工作中，对偶理论在精确算法设计中经常被用到；在鲁棒优化（Robust Optimization）等涉及多层规划（Multilevel Programming）的问题中，也有非常广泛的应用，很多时候可以化腐朽为神奇。尤其在鲁棒优化中，有些问题可以巧妙地将内层（inner level）的模型转化成 LP，从而可以通过对偶，将双层（bi-level）的模型转化成单阶段（single level）的模型，进而用单层模型的相关算法求解鲁棒优化问题。

本书主要探讨在实际的科研当中遇到的一些稍微复杂一点的 LP，如何写出其对偶问题？实际上在一些学术期刊中，例如 *Operations Research*, *Transportation Science* 等期刊上比较近期的文章，也时不时会看到这样的操作。这个操作其实并不是抬手就能实现的，很多时候需要反复修改，才能将对偶问题正确地写出来。

先来看一个比较容易的线性规划问题：

$$\begin{aligned}
\max \quad & Z = 2x_1 + 3x_2 && && && (4.1)\\
\text{s.t.} \quad & 5x_1 + 4x_2 \leqslant 170 && \rightarrow && y_1 && (4.2)\\
& 2x_1 + 3x_2 \leqslant 100 && \rightarrow && y_2 && (4.3)\\
& x_1, x_1 \geqslant 0 && && && (4.4)
\end{aligned}$$

其对偶问题比较容易写出：

$$\begin{aligned}
\min \quad & W = 170y_1 + 100y_2 && (4.5)\\
\text{s.t.} \quad & 5y_1 + 2y_2 \geqslant 2 && (4.6)\\
& 4y_1 + 3y_2 \geqslant 3 && (4.7)\\
& y_1, y_1 \geqslant 0 && (4.8)
\end{aligned}$$

关于对偶问题的理解

假设上述原问题和对偶问题中，原问题中 x_1 和 x_2 分别代表一个家具生产公司生产桌子和椅子的数量，目标函数系数为出售桌子和椅子获得的利润。两条约束分别是某两种生产原材料的当前可用量。

原问题就是，自己利用这些生产原材料，制造桌子和椅子，决策出生产多少张桌子和多少把椅子，使得总利润最大化。

而对偶问题的思路就完全是另一个角度。如果该厂决定不自己生产，而是将原材料卖给其他生产商生产。此时，该厂需要决策，每一种原材料卖多少钱合适？卖得太高，可能不会有人买，卖得太低，还不如自己生产获得的利润高。那么能不能找到一个最低的定价标准，使得卖了这些原材料获得的钱，正好与自己生产能够赚的利润一样多？这样，该厂只需要参照这个最低定价标准来定价，就能保证自己不吃亏。

该问题中，原问题为 max，而对偶问题为 min，如果原问题和对偶问题都是可行的，那么对偶问题的目标函数一定是大于或等于原问题的。也就是说，以很高的价格卖出原材料，获得的利润会比自己生产获得的利润高。但是当不断降低卖出原材料的价格，最终会找到一个原材料的定价方案，使得卖出原材料和自己生产获得的利润一样高。这也正是对偶问题的解。很多其他问题，也可以用类似的方法去理解，比如最短路问题等。用这种方法更容易理解对偶问题的含义。

4.2 原问题与对偶问题之间的关系

当原问题不是标准形时，可以借助表 4.1写出原问题的对偶（《运筹学》教材编写组，2012），或者可以转成标准形再写对偶。

表 4.1 原问题和对偶问题的关系

原问题（对偶问题）	对偶问题（原问题）
目标函数为 max	目标函数为 min
变量 $\begin{cases} n\text{个} \\ \geqslant 0 \\ \leqslant 0 \\ \text{无约束} \end{cases}$	$\left.\begin{matrix} n\text{个} \\ \geqslant \\ \leqslant \\ = \end{matrix}\right\}$ 约束条件
目标函数中变量的系数	约束条件的右端项
约束条件 $\begin{cases} m\text{个} \\ \leqslant \\ \geqslant \\ = \end{cases}$	$\left.\begin{matrix} m\text{个} \\ \geqslant 0 \\ \leqslant 0 \\ \text{无约束} \end{matrix}\right\}$ 变量
约束条件右端项	目标函数中变量的系数

表 4.1 用于检查比较方便，但是用于写对偶问题可能有些烦琐。因此，可以首先把原问题全部化成统一的形式。如下：

（1）max 问题，变量都 $\geqslant 0$，约束都 $\leqslant 0$（粗略理解为：资源都是有限的）。

（2）min 问题，变量都 $\geqslant 0$，约束都 $\geqslant 0$（粗略理解为：需求至少要被满足）。

这样 max 问题和 min 问题就可以比较容易对应，当然也可以更容易写出对偶问题。上面的小例子就是一个这样的示范。

在化简成标准形式时，我们可以用线性规划的矩阵形式更方便地记忆原问题和对偶问题之间的关系。

原问题为

$$\max \quad Z = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \tag{4.9}$$

$$\text{s.t.} \quad \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b} \tag{4.10}$$

$$\boldsymbol{x} \geqslant \boldsymbol{0} \tag{4.11}$$

其中 $\boldsymbol{c}$ 是一个 n 维列向量，$\boldsymbol{c}^{\mathrm{T}}=(c_1, c_2, \cdots, c_n)$，$\boldsymbol{x}$ 是一个 n 维列向量，$\boldsymbol{x}^{\mathrm{T}}=(x_1, x_2, \cdots, x_n)$。且 $\boldsymbol{c}, \boldsymbol{x} \in \mathbb{R}^n$，$\boldsymbol{A} \in \mathbb{R}^{m \times n}$，是上述线性规划的约束系数矩阵（$m$ 个约束，n 个决策变量）。$\boldsymbol{b} \in \mathbb{R}^m$ 也是一个列向量，是约束的右端项。

注意：一般来讲，决策变量 $\boldsymbol{x}$ 都用列向量来表示。

以上面的两个变量的线性规划为例，我们有

$$\boldsymbol{c} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \boldsymbol{A} = \begin{bmatrix} 5 & 4 \\ 2 & 3 \end{bmatrix}, \boldsymbol{b} = \begin{bmatrix} 170 \\ 100 \end{bmatrix}$$

上面线性规划的对偶问题的矩阵形式为

$$\min \quad W = \boldsymbol{b}^{\mathrm{T}}\boldsymbol{y} \tag{4.12}$$

$$\text{s.t.} \quad \boldsymbol{A}^{\mathrm{T}}\boldsymbol{y} \geqslant \boldsymbol{c} \tag{4.13}$$

$$\boldsymbol{y} \geqslant \boldsymbol{0} \tag{4.14}$$

我们来验证一下，在对偶问题中，确实是一一对应的。$\boldsymbol{b}^{\mathrm{T}}\boldsymbol{y} = 170y_1 + 100y_2$，约束系数矩阵恰好为 $\boldsymbol{A}^{\mathrm{T}}$。矩阵形式可以更方便我们理解对偶问题和原问题之间的关系。

关于对偶理论，有一些相关定理，如对偶定理、强对偶定理等。对于这些内容，我们在之后的 Benders 分解算法等部分会频繁用到。接下来，我们就对这些相关理论略作介绍。

4.3 对偶理论相关重要定理

首先用矩阵的形式给出线性规划的解的一些重要公式。

$$\max \quad z = \begin{bmatrix} \boldsymbol{c}_B^{\mathrm{T}} & \boldsymbol{c}_N^{\mathrm{T}} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_B \\ \boldsymbol{x}_N \end{bmatrix} \tag{4.15}$$

$$\text{s.t.} \quad \begin{bmatrix} B & N \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_B \\ \boldsymbol{x}_N \end{bmatrix} = b \tag{4.16}$$

对于模型的约束 (4.16)，我们有

$$B\boldsymbol{x}_B + N\boldsymbol{x}_N = b$$

两边左乘 B^{-1} 有

$$B^{-1}B\boldsymbol{x}_B + B^{-1}N\boldsymbol{x}_N = B^{-1}b$$
$$\boldsymbol{x}_B = B^{-1}b - B^{-1}N\boldsymbol{x}_N$$

在解中，令所有非基变量均为 0，即

$$\boldsymbol{x}_N = \boldsymbol{0}$$

其中，$\boldsymbol{0}$ 是一个维数为非基变量个数的零向量。因此根据上述表达式，可以得到模型的解。

接下来，模型的目标函数值为

$$\begin{aligned} z &= \begin{bmatrix} \boldsymbol{c}_B^{\mathrm{T}} & \boldsymbol{c}_N^{\mathrm{T}} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_B \\ \boldsymbol{x}_N \end{bmatrix} \\ &= \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{x}_B + \boldsymbol{c}_N^{\mathrm{T}}\boldsymbol{x}_N \\ &= \boldsymbol{c}_B^{\mathrm{T}}\left(B^{-1}b - B^{-1}N\boldsymbol{x}_N\right) + \boldsymbol{c}_N^{\mathrm{T}}\boldsymbol{x}_N \\ &= \boldsymbol{c}_B^{\mathrm{T}}B^{-1}b + \left(\boldsymbol{c}_N^{\mathrm{T}} - \boldsymbol{c}_B^{\mathrm{T}}B^{-1}N\right)\boldsymbol{x}_N \end{aligned}$$

上面的表达式中，$\boldsymbol{c}_N^{\mathrm{T}} - \boldsymbol{c}_B^{\mathrm{T}}B^{-1}N$ 就是检验数（reduced cost）。

然后考虑下面的主问题和对偶问题。

表 4.2 原问题与对偶问题

原 问 题			对 偶 问 题		
max	$Z = \sum_{j=1}^{n} c_j x_j$		min	$W = \sum_{i=1}^{m} b_i y_i$	
s.t.	$\sum_{j=1}^{n} a_{ij}x_j \leqslant b_i,$	$\forall i = 1,2,\cdots,m$	s.t.	$\sum_{i=1}^{m} a_{ij}y_i \geqslant c_j$	$\forall j = 1,2,\cdots,n$
	$x_j \geqslant 0,$	$\forall j = 1,2,\cdots,n$		$y_i \geqslant 0,$	$\forall i = 1,2,\cdots,m$
原问题（矩阵形式）			对偶问题（矩阵形式）		
max	$Z = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}$		min	$W = \boldsymbol{y}^{\mathrm{T}}\boldsymbol{b}$	
s.t.	$A\boldsymbol{x} \leqslant \boldsymbol{b}$		s.t.	$A^{\mathrm{T}}\boldsymbol{y} \geqslant \boldsymbol{c}$	
	$\boldsymbol{x} \geqslant \boldsymbol{0}$			$\boldsymbol{y} \geqslant \boldsymbol{0}$	

其中，$\boldsymbol{c}, \boldsymbol{y}, \boldsymbol{x}$ 均是列向量。并且在任一次的单纯形表迭代中，单纯形表的第 0 行（row 0）都是下面的形式。

表 4.3 任意一次单纯形算法迭代中单纯形表中的第 0 行

Iter	Basic var	Eq.	Coefficient of									RHS
			Z	x_1	x_2	$\cdots$	x_n	x_{n+1}	x_{n+2}	$\cdots$	x_{n+m}	
Any	Z	(0)	1	z_1-c_1	z_2-c_2	$\cdots$	z_n-c_n	y_1	y_2	$\cdots$	y_m	W

注意：这里有一点很重要，原问题表的 row 0 中，初始变量（original variable）的系数 z_j-c_j 相当于对偶问题的剩余变量（具体解释见下文）。当原问题没有达到最优时，row 0 中一定有检验数为正（也就是 $z_j-c_j<0$）的情况，也就是对偶问题的剩余变量为负，这说明此时对偶问题不可行。

另外，需要明确下面几点对应关系。

（1）变量 x_i 的检验数和 row 0 中信息的对应关系为

$$\text{reduced cost}_j=-(z_j-c_j) \tag{4.17}$$

（2）z_j-c_j 的值就对应对偶问题第 j 条约束的剩余变量（surplus variable）（或者松弛变量（slack variable），对应 min 问题）。

对于第（2）条对应关系，可以用下面的方法来理解，将检验数的相反数的表达式改写为

$$z_j-c_j=\sum_{i=1}^{m}a_{ij}y_i-c_j \tag{4.18}$$

$$=-\Big(c_j-\sum_{i=1}^{m}a_{ij}y_i\Big),\quad \forall j=1,2,\cdots,n \tag{4.19}$$

可以清楚地看到，$\sum\limits_{i=1}^{m}a_{ij}y_i-c_j$ 就是对偶问题的第 j 条约束，$\sum\limits_{i=1}^{m}a_{ij}y_i\geqslant c_j$ 对应的剩余变量。而

$$c_j-\sum_{i=1}^{m}a_{ij}y_i,\qquad \forall j=1,2,\cdots,n \tag{4.20}$$

就正好是变量 x_j 的检验数（reduced cost）。

因此，对偶问题的每一个 corner-point 解 $(y_1,y_2,\cdots,y_m)$ 以及 z_j-c_j 都会产生一个对偶问题的基解 $(y_1,y_2,\cdots,y_m,z_1-c_1,z_2-c_2,\cdots,z_n-c_n)$。根据对偶问题的标准型，对偶问题有 n 个约束和 $n+m$ 个变量（其中有 n 个剩余变量）。因此，对偶问题的一个基解含有 n 个基变量和 m 个非基变量。

下面给出几个性质。

定理 4.3.1 互补基解性质（**Complementary basic solutions property**）（Hillier, 2012）：原问题的一个基解，都对应产生对偶问题的一个互补的基解。且这两个基解对应的目标函数是一样的（$Z=W$，但这两个基解并不一定是可行的）。在给定了单纯形表 row 0 的信息之后，可以得到互补的对偶问题的基解为 $(y,z-c)$。

定理 4.3.2 互补最优基解性质（**Complementary optimal basic solutions property**）（Hillier，2012）：原问题的一个最优基解，对应产生对偶问题的一个最优基解。且这两个最优基解对应的目标函数是一样的（$Z = W$）。在给定了单纯形表 row 0 的信息之后，可以得到互补的对偶问题的最优基解为 $(\boldsymbol{y}^*, \boldsymbol{z}^* - \boldsymbol{c})$。

下面给出原问题的最优性条件。

引理 4.3.1（Hillier，2012） 原问题的最优性条件（也就是原问题存在最优解的条件）为

$$z_j - c_j \geqslant 0, \quad \forall j = 1, 2, \cdots, n \tag{4.21}$$

$$y_i \geqslant 0, \quad \forall i = 1, 2, \cdots, m \tag{4.22}$$

（1）条件 (4.21) 表示，在 max 问题的 row 0 中，初始变量的检验数为负，已经没有可以提升的空间；另外，也说明对偶问题的所有约束的松弛变量取值均大于或等于 0。

（2）条件 (4.22) 中，y_i 其实就是对偶问题的解，这表示，对偶问题的解也必须可行（即满足非负性，且 $z_j - c_j \geqslant 0$ 保证了对偶问题的约束也被满足）。

另外还有几个相关的定理。

定理 4.3.3 弱对偶性（**Weak duality property**）（Hillier，2012）：如果 $\boldsymbol{x}$ 是原问题（最大化问题）的一个可行解，并且 $\boldsymbol{y}$ 是对偶问题的一个可行解，则

$$\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \leqslant \boldsymbol{y}^{\mathrm{T}}\boldsymbol{b} \tag{4.23}$$

证明：对于表 4.2中原问题的约束 $\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}$，左乘向量 $\boldsymbol{y}^{\mathrm{T}}$，则有

$$\boldsymbol{y}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{y}^{\mathrm{T}}\boldsymbol{b} \tag{4.24}$$

对于对偶问题的约束 $\boldsymbol{y}^{\mathrm{T}}\boldsymbol{A} \geqslant \boldsymbol{c}^{\mathrm{T}}$，右乘 $\boldsymbol{x}$，则有

$$\boldsymbol{y}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x} \geqslant \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \tag{4.25}$$

综合以上，有

$$z_{\mathrm{Primal}} = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \leqslant \boldsymbol{y}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{y}^{\mathrm{T}}\boldsymbol{b} = z_{\mathrm{Dual}} \tag{4.26}$$

因此，命题得证。

定理 4.3.4 强对偶性（**Strong duality property**）（Hillier，2012）：如果 $\boldsymbol{x}^*$ 是原问题（最大化问题）的一个最优解，并且 $\boldsymbol{y}^*$ 是对偶问题的一个最优解，则

$$\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}^* = (\boldsymbol{y}^*)^{\mathrm{T}}\boldsymbol{b} \tag{4.27}$$

证明：由之前的推导，可以得到原问题的最优值为

$$z = \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{b} + \left(\boldsymbol{c}_N^{\mathrm{T}} - \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{N}\right)\boldsymbol{x}_N$$

如果原问题的解达到了最优，则所有非基变量的检验数都小于或等于 0。也就是对于所有的非基变量，都满足：

$$c_N^{\mathrm{T}} - c_B^{\mathrm{T}} B^{-1} N \leqslant 0 \Longrightarrow c_B^{\mathrm{T}} B^{-1} N \geqslant c_N^{\mathrm{T}}$$

另外，有

$$c_B^{\mathrm{T}} B^{-1} B = c_B^{\mathrm{T}}$$

综合以上，有

$$c_B^{\mathrm{T}} B^{-1} \begin{bmatrix} B & N \end{bmatrix} \geqslant \begin{bmatrix} c_B^{\mathrm{T}} & c_N^{\mathrm{T}} \end{bmatrix}$$

即

$$c_B^{\mathrm{T}} B^{-1} A \geqslant c^{\mathrm{T}}$$

等价于

$$y^{\mathrm{T}} A \geqslant c^{\mathrm{T}}$$

这说明，当原问题达到最优解时，对应的对偶问题的解 y^* 同时是对偶问题的可行解。

接下来只需要证明此时原问题的目标函数和对偶问题的目标函数值相同。

由 $z = c_B^{\mathrm{T}} B^{-1} b + \left(c_N^{\mathrm{T}} - c_B^{\mathrm{T}} B^{-1} N\right) x_N$ 可得，原问题得到最优解时，非基变量 x_N 取值均为 $\mathbf{0}$，因此原问题的目标值为

$$\begin{aligned} z_{\text{Primal}} &= c_B^{\mathrm{T}} B^{-1} b \\ &= y^{\mathrm{T}} b \\ &= z_{\text{Dual}} \end{aligned}$$

综上，原问题取得最优解时，对应的对偶问题的解是可行解。并且此时原问题的目标函数和对偶问题的目标函数相同。因此强对偶定理得证。

定理 4.3.5 互补松弛性（**Complementary slackness property**）(Hillier, 2012)：*如果 x^* 是原问题的一个最优解，对应的松弛变量值为 x_s；并且 y^* 是对偶问题的一个最优解，对应的松弛变量值为 y_s；则*

$$x_s^{\mathrm{T}} y^* = 0 \tag{4.28}$$

$$(x^*)^{\mathrm{T}} y_s = 0 \tag{4.29}$$

证明：我们分情况讨论。

第 1 种情况，如果最优解中松弛变量 $x_{s_i} > 0$，则 x_{s_i} 是基变量，其检验数一定为 0，即

$$\text{reduced cost}_{s_i} = 0 - c_B^{\mathrm{T}} B^{-1} \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$= -\boldsymbol{y}^{\mathrm{T}} \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$= 0$$

其中，最右端的列向量是最优单纯形表中松弛变量 x_{s_i} 对应的列。由于 x_{s_i} 是基变量，因此该列的系数组成的向量一定是单位向量。不失一般性，我们假设第一个元素为 1。其他位置为 1 的情况也相同。

由上式，有 $(\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1})_i \times 1 = 0$，即 $y_i = 0$。

第 2 种情况，如果最优解中松弛变量 $x_{s_i} \leqslant 0$，而松弛变量又为非负数，因此 $x_{s_i} = 0$。

综合上述 2 种情况，均有 $x_{s_i} \cdot y_i = 0$。同理，对于对偶问题，该结论也成立。因此，互补松弛性定理得证。

关于互补松弛性的理解

对偶变量，或者影子价格，其实反映了增加一个单位的资源导致目标函数能够增加或减少的量。如果说一条约束对应的松弛变量为正数，这说明该资源非常充足，甚至有剩余。此时，增加一个单位该资源的供应，也不会使目标函数得到改善，因此，其影子价格（对偶变量）取值一定为 0，原问题的松弛变量和约束对应的对偶变量乘积为 0。如果原问题一个约束的松弛变量为 0，说明该资源紧缺，已经全部用完，增加该资源的供应，很有希望会使得目标函数得到改善，因此对偶变量（影子价格）大于或等于 0。

综合两种情况，原问题的松弛变量和约束对应的对偶变量的乘积为 0。

定理 4.3.6 互补解性质（**Complementary solutions property**）(Hillier, 2012)：在每次单纯形算法迭代中，单纯形算法都会同时找到一个原问题的 CPF (corner point feasible) 的解 $\boldsymbol{x}$，以及一个对偶问题的互补解 $\boldsymbol{y}$（就是 row 0 中，原问题松弛变量的系数），并且满足此时根据二者的值计算出的原问题和对偶问题的目标函数值相等，即

$$\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} = \boldsymbol{y}^{\mathrm{T}}\boldsymbol{b} \tag{4.30}$$

如果此时，$\boldsymbol{x}$ 不是原问题的最优解，则 $\boldsymbol{y}$ 也不是对偶问题的可行解。

也就是说，只有当 $\boldsymbol{x}$ 是原问题的最优解时，$\boldsymbol{y}$ 才是对偶问题的可行解，当然同时也是对偶问题的最优解。

定理 4.3.7 互补最优解性质（**Complementary optimal solutions property**）(Hillier, 2012)：在最后一次的迭代中，单纯形算法同时得到了原问题的一个最优解 $\boldsymbol{x}^*$ 以及对偶问题的一个互补最优解 $\boldsymbol{y}^*$（就是 row 0 中，原问题松弛变量的系数），并且有

$$c^{\mathrm{T}}x^{*}=(y^{*})^{\mathrm{T}}b \tag{4.31}$$

此时 y^{*} 其实就是原问题对应约束的影子价格（shadow price）。

下面是对偶理论的内容。

定理 4.3.8 对偶定理（Duality theorem）（Hillier，2012）：原问题和对偶问题解之间的关系只有下面的几种可能。

（1）如果一个问题有可行解，并且目标函数是有界的（因此也有最优解），那么另一个问题也有可行解和有界的目标函数以及最优解，此时强对偶理论和弱对偶理论都是成立的。

（2）如果一个问题有可行解但是目标函数无界（也就是没有最优解），那么另一个问题无可行解（即原问题无界，对偶问题无可行解）。

（3）如果一个问题没有可行解，那么另一个问题要么无可行解，要么无界（原问题无可行解，对偶问题无可行解或者无界）。

对于定理中的第（2）点，目标函数无界，就说明永远存在一个变量的检验数为正（max 问题），使得算法结束不了。原问题表的 row 0 中，初始变量的系数 z_j-c_j 相当于对偶问题的剩余变量（surplus variable）。根据上述论述，对于 max 问题，当原问题没有达到最优解时，row 0 中一定有检验数为正（也就是 $z_j-c_j<0$ 的情况），也就是对偶问题的剩余变量为负，这说明此时对偶问题不可行。

对于定理中的第（3）点，我们在之后的部分再进行解释。

下面是对偶理论的进一步分析理解。

（1）原问题无界，则对偶问题无可行解。

假设原问题为 max 问题，根据上面的理论，原问题的最优性条件为

$$\begin{aligned} z_j-c_j&\geqslant 0, \quad \forall j=1,2,\cdots n\\ y_i&\geqslant 0, \quad \forall i=1,2,\cdots m \end{aligned}$$

如果原问题无界，则说明总存在一个 $z_j-c_j<0$，使得目标函数可以增加，因此单纯形表迭代停不下来，因此对偶问题无解。因为对偶问题的约束就是

$$\min \quad W=\sum_{i=1}^{m}b_iy_i=\boldsymbol{y}^{\mathrm{T}}\boldsymbol{b} \tag{4.32}$$

$$\sum_{i=1}^{m}a_{ij}y_i\geqslant c_j, \quad \forall j=1,2,\cdots,n \tag{4.33}$$

$$y_i\geqslant 0, \qquad \forall i=1,2,\cdots,m \tag{4.34}$$

由于约束 i 限制 $\sum\limits_i y_ia_{ij}\geqslant c_j$，我们可以得出

$$\sum_i a_{ij}y_i-c_j=\boldsymbol{y}^{\mathrm{T}}\boldsymbol{A}_{\cdot j}-c_j=\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{A}_j-c_j=z_j-c_j\geqslant 0$$

因此，就可以按照下面的逻辑推理：原问题无界，从而得到总存在 $z_j - c_j < 0$，从而总存在一些约束应该是对 i 求和 $\sum_i y_i a_{ij} \geqslant c_j$ 不能被满足，因此对偶问题非可行。

（2）原问题无可行解，则对偶问题或者无可行解，或者无界。

根据对偶问题和原问题的对称性，若原问题无界，则对偶问题无可行解。由此可知，若原问题无可行解，则对偶问题可能是无界（有可行解）的。

另外一种情况就是原问题和对偶问题都非可行。

因此原问题无可行解，则对偶问题无界或者非可行。

4.4 最短路问题的对偶

对于比较简单的线性规划问题，我们不难写出其对偶问题，但是对于形式稍微复杂一些的线性规划，写出其对偶问题就不是很容易了。本节我们就以最短路问题（SPP）和多商品网络流问题（MCNF）为例，讲解一种比较方便地写出大规模线性规划对偶的方法：Excel+具体小算例。

首先我们来看 SPP 的模型

$$\min \quad \sum_{e \in A} d_e x_e \tag{4.35}$$

$$\text{s.t.} \quad \sum_{e \in \text{out}(i)} x_e - \sum_{e \in \text{in}(i)} x_e = \begin{cases} 1, & i = s \\ -1, & i = t \\ 0, & \text{其他} \end{cases} \tag{4.36}$$

$$0 \leqslant x_e \leqslant 1, \qquad \forall e \in A \tag{4.37}$$

这里花括号里有几个条件判断，直接通过观察就写出该问题的对偶有些困难。

注意： 最短路问题即使将整数约束松弛掉，仍然存在最优整数解，因此最短路问题可以转化成等价的线性规划，只有线性规划才可以用本节介绍的方法进行对偶。实际上还存在其他对偶方法，但本书不再进行详细介绍。

4.4.1 借助 Excel 和具体小算例写出大规模 SPP 的对偶

首先我们来看一个比较小的算例。该算例来自文献（Cappanera and Scaparra，2011）。网络结构图和参数如图 4.1所示。

我们根据最短路问题的模型，将上面的小算例的具体形式写到 Excel 表格中，如图 4.2所示。

我们把这个表起个名字，叫原问题表（Primal tabular）。其中，每一列代表一个决策变量 $x_{ij}, \forall (i,j) \in A$。表中其余的部分分别如下：

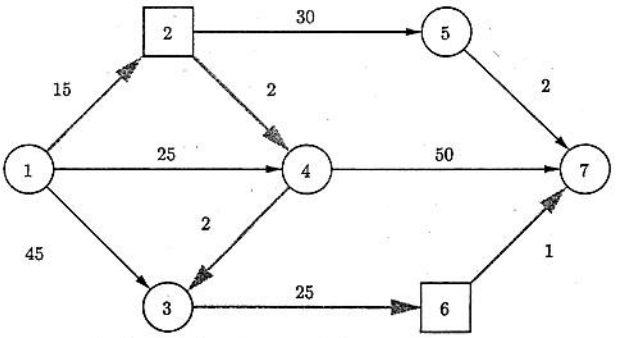


图 4.1 最短路问题：示例网络

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	distance													
2	min	x_1_2	x_1_4	x_1_3	x_2_5	x_2_4	x_4_7	x_4_7	x_4_3	x_3_6	x_6_7		RHS	Dual Var
3	orig_1	1	1	1								=	1	pi_1
4	inter_2	-1			1	1						=	0	pi_2
5	inter_3			-1					-1	1		=	0	pi_3
6	inter_4		-1			-1		1	1			=	0	pi_4
7	inter_5				-1		1					=	0	pi_5
8	inter_6									-1	1	=	0	pi_6
9	des_7						-1	-1			-1	=	-1	pi_7

图 4.2 SPP 模型：表格形式

（1）第 1 行代表该条 (i,j) 的距离；

（2）第 2 行代表变量 $x_{ij}, \forall (i,j) \in A$；

（3）第 1 行和第 2 行就组成了目标函数 $\sum_{e \in A} d_e x_e$；

（4）第 3~9 行代表每个节点 $\forall i \in V$ 的约束；

（5）最后 1 列代表每个约束的对偶变量（Dual variable）。

完成上面的工作以后，接下来就是最关键的部分。按照对偶的方法，进行下面的操作：

（1）将原问题表的 RHS 列和对偶变量列复制，转置成 2 行，放在一个新表格（我们给它起名叫对偶问题表（Dual tabular））的头 2 行；

（2）然后将原问题表的整个约束系数矩阵复制，转置到对偶问题表头 2 行下面；

（3）再把原问题表的 distance 行和 min 行复制，转置，放在对偶问题表的最右面 2 列；

（4）最后把对偶问题表中改成 max；

（5）为了明确对偶问题中各个变量的符号（正负性）以及每个约束的符号（relation），在对偶问题表中加入 1 行（就是第 3 行），表示变量的符号。同时在对偶问题表约束矩阵后加入 1 列，表示约束的符号。

完成上面的所有操作后，对偶问题表就是如图 4.3所示的样式。

	A	B	C	D	E	F	G	H	I	J
1	RHS	1	0	0	0	0	0	−1		
2	Dual Var	pi_1	pi_2	pi_3	pi_4	pi_5	pi_6	pi_7		
3	max	=	=	=	=	=	=	=		
4	x_1_2	1	−1						<=	15
5	x_1_4	1			−1				<=	25
6	x_1_3	1		−1					<=	45
7	x_2_5		1			−1			<=	30
8	x_2_4		1		−1				<=	2
9	x_5_7					1		−1	<=	2
10	x_4_7				1			−1	<=	50
11	x_4_3			−1	1				<=	2
12	x_3_6			1			−1		<=	25
13	x_6_7						1	−1	<=	1

图 4.3 SPP 模型的对偶问题：表格形式

按照上面关系图中的信息，我们可以确定，对偶变量 π_i 都是无约束的，我们用 $=$ 表示，对偶问题中的约束都是 $\leqslant$ 的。这样，对偶就完成了。

但是，这还是一个具体算例的对偶，需要将这个具体的算例通过提取信息并整理，将其写成一个通用的公式形式。

观察图 4.3，每行都对应一条弧 $(i,j) \in A$，例如第 1 行是 $(1,2)$，第 2 行是 $(1,4)$ 等。可以看到，对应出发点的变量系数全是 1，对应终点的系数全是 -1，无一例外，因此可以断定，这个约束可以写为如下形式：

$$\pi_i - \pi_j \leqslant d_{ij}, \quad \forall (i,j) \in A$$

结合目标函数以及变量的符号，可以写出 SPP 的对偶问题的数学表达式形式如下：

$$\max \quad \pi_s - \pi_t \tag{4.38}$$

$$\text{s.t.} \quad \pi_i - \pi_j \leqslant d_{ij}, \quad \forall (i,j) \in A \tag{4.39}$$

$$\pi_i \text{ free}, \quad \forall i \in V \tag{4.40}$$

此时，SPP 的对偶问题就完成了。但是还有一些小的地方需要注意，根据文献（Cappanera and Scaparra，2011），我们还能稍微做一些小拓展。

首先把所有对偶变量取相反数，等价转化为

$$\max \quad \pi_t - \pi_s \tag{4.41}$$

$$\text{s.t.} \quad \pi_j - \pi_i \leqslant d_{ij}, \quad \forall (i,j) \in A \tag{4.42}$$

$$\pi_i \text{ free}, \quad \forall i \in V \tag{4.43}$$

因为 π_i 是无约束（free）的，因此上述转化是等价的。也正由于 π_i 是无约束的，且是最大化 $\pi_s - \pi_t$，因此可以直接把 π_s 设置成 0，也就是约束 $\pi_s = 0$，这样并不会改变最优解。当然，仍然保持目标函数为 max，就可以把模型等价转化为

$$\max \quad \pi_t \tag{SPP-Dual}$$

$$\text{s.t.} \quad \pi_j - \pi_i \leqslant d_{ij}, \qquad \forall (i,j) \in A \tag{4.44}$$

$$\pi_s = 0 \tag{4.45}$$

$$\pi_i \ \text{ free}, \qquad \forall i \neq s, i \in V \tag{4.46}$$

这样就能保证所有的对偶变量 π_i 都是非负的，这个模型与原模型完全等价，而且问题复杂度相同。

依据这个模型，可以对对偶问题进行一个比较直观的理解了。

> 就像生产计划问题的对偶问题，可以按照出租的逻辑来理解一样，SPP 问题的原问题是一个客户要在网络上从起点到终点走一条最短路线，花销最少。那么 SPP 的对偶问题就可以理解为，从网络运营者的角度，就是要通过在每个节点设置收费金额，来看看能获得的收益的下限。即运营者要获得利润，就需要在每个节点设置收费金额（或者惩罚金额），并且设置完成后，运营者想要知道该种设置方案最低能获得多少收益（即使用者走最短路的情况下，运营者的收益是多少。当然，使用者不走最短路，运营者的收益会更大）。关于惩罚的设置也有一个硬性要求，即在一条边连接的两个节点上，惩罚的差值不能超过这条边的权重。在每个节点上的惩罚金额，实际上就可以看作到达该点的最小费用。最终通过考查终点 t 的惩罚金额 π_t，我们就可以得知该网络的最少收益是多少了（即 π_t 的取值）。

4.4.2 SPP 中存在负环的特例

根据上面的例子，这里写出了 SPP 的对偶问题，但是还有一些情况需要注意，就是如果 SPP 的网络中存在负环，其对偶问题就会无解。看下面这个带有负环的例子，如图 4.4 所示。

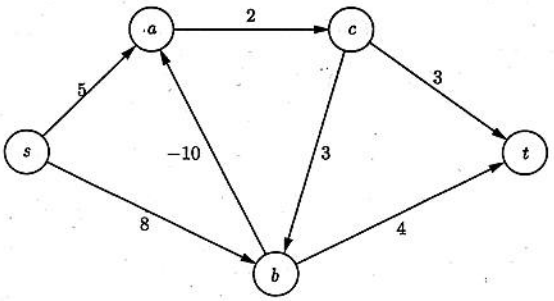


图 4.4 最短路问题：网络中存在负环的情况

这里用 Python 调用 Gurobi 求解该算例的最优解，代码如下：

SPPExampleNegativeCycle.py

```
from gurobipy import *
import pandas as pd
import numpy as np

Nodes = ['s', 'a', 'b', 'c', 't']

Arcs = {('s','a'): 5
        ,('s','b'): 8
        ,('a','c'): 2
        ,('b','a'): -10
        ,('c','b'): 3
        ,('b','t'): 4
        ,('c','t'): 3
       }

model = Model('dual problem')

# add decision variables
X = {}
for key in Arcs.keys():
    index = 'x_' + key[0] + ',' + key[1]
    X[key] = model.addVar(vtype=GRB.BINARY
                          , name= index
                         )

# add objective function
obj = LinExpr(0)
for key in Arcs.keys():
    obj.addTerms(Arcs[key], X[key])

model.setObjective(obj, GRB.MINIMIZE)

# constraint 1 and constraint 2
lhs_1 = LinExpr(0)
lhs_2 = LinExpr(0)
for key in Arcs.keys():
    if(key[0] == 's'):
        lhs_1.addTerms(1, X[key])
    elif(key[1] == 't'):
        lhs_2.addTerms(1, X[key])
model.addConstr(lhs_1 == 1, name = 'start flow')
```

```
model.addConstr(lhs_2 == 1, name = 'end flow')

# constraints 3
for node in Nodes:
    lhs = LinExpr(0)
    if(node != 's' and node != 't'):
        for key in Arcs.keys():
            if(key[1] == node):
                lhs.addTerms(1, X[key])
            elif(key[0] == node):
                lhs.addTerms(-1, X[key])
    model.addConstr(lhs == 0, name = 'flow conservation')

model.write('model_spp.lp')
model.optimize()

print(model.ObjVal)
for var in model.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
```

求解结果如下：

SPPExampleNegativeCycle.py

```
Solution count 1: 3

Optimal solution found (tolerance 1.00e-04)
Best objective 3.000000000000e+00, best bound 3.000000000000e+00, gap
       0.0000%
3.0
x_s,b    1.0
x_a,c    1.0
x_b,a    1.0
x_c,t    1.0
```

可以看到，最短路为 $s \to b \to a \to c \to t$，路径长度为 3。

但是如果我们用求解器求解其对偶问题，根据模型 (SPP-Dual) 写出代码如下：

SPPExampleNegativeCycle.py

```
model = Model('dual problem')

pi_a = model.addVar(lb=-1000, ub=1000, vtype=GRB.CONTINUOUS, name= "pi_a")
pi_b = model.addVar(lb=-1000, ub=1000, vtype=GRB.CONTINUOUS, name= "pi_b")
```

```
pi_c = model.addVar(lb=-1000, ub=1000, vtype=GRB.CONTINUOUS, name= "pi_c")
pi_s = model.addVar(lb=0, ub=0, vtype=GRB.CONTINUOUS, name= "pi_s")
pi_t = model.addVar(lb=-1000, ub=1000, vtype=GRB.CONTINUOUS, name= "pi_t")

obj = LinExpr(0)
obj.addTerms(1 , pi_t)
model.setObjective(obj, GRB.MAXIMIZE)

# lhs relation , rhs
model.addConstr(pi_a - pi_s <= 5)
model.addConstr(pi_b - pi_s <= 8)
model.addConstr(pi_t - pi_c <= 3)
model.addConstr(pi_t - pi_b <= 4)
model.addConstr(pi_a - pi_b <= -10)
model.addConstr(pi_c - pi_a <= 2)
model.addConstr(pi_b - pi_c <= 3)

model.write('model2.lp')
model.optimize()

print(model.ObjVal)
for var in model.getVars():
    print(var.varName, '\t', var.x)
```

求解结果如下：

SPPExampleNegativeCycle.py

```
Gurobi Optimizer version 9.0.1 build v9.0.1rc0 (win64)
Optimize a model with 7 rows, 5 columns and 14 nonzeros
Model fingerprint: 0xd51f0a26
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [1e+00, 1e+00]
  Bounds range     [1e+03, 1e+03]
  RHS range        [2e+00, 1e+01]
Presolve time: 0.00s
Presolved: 7 rows, 5 columns, 14 nonzeros

Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    2.0100000e+03   5.026250e+02   0.000000e+00      0s

Solved in 2 iterations and 0.01 seconds
Infeasible model
```

可以看到结果为不可行。这是因为到一个点 i 的最小费用为 π_i，同一条弧 (i,j) 上的两个点的费用差值必须满足 $\pi_j - \pi_i \leqslant d_{ij}$，如果 d_{ij} 为负，会导致同一个点 i 对应多个最小费用 π_i，问题当然会变得不可行。

如果将 (b,a) 的权重修改为 10，如图 4.5 所示。

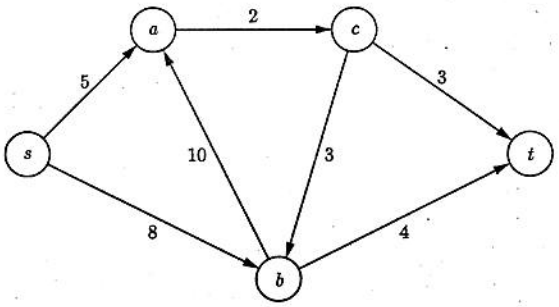


图 4.5 最短路问题：通过修改边的权重，将负环删去

相应地将代码中的对应约束改成如下形式：

SPPExampleNegativeCycle.py

```
# model.addConstr(pi_a - pi_b <= -10)
model.addConstr(pi_a - pi_b <= 10)
```

重新求解模型，可以得到结果如下：

SPPExampleNegativeCycle.py

```
Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0   1.0000000e+01   2.501250e+02   0.000000e+00      0s
       2   1.0000000e+01   0.000000e+00   0.000000e+00      0s

Solved in 2 iterations and 0.02 seconds
Optimal objective  1.000000000e+01
10.0
pi_s      0.0
pi_a      5.0
pi_b      6.0
pi_c      7.0
pi_t      10.0
```

这次的最优解为 $s \to a \to c \to t$，路径长度为 10。

我们也发现，当设置 $\pi_s = 0$ 时，$\pi_i, i \in \{s,a,b,d,t\}$ 的取值，正好是从起点 s 出发到达点 i 的最短距离。这正好印证了之前的解释，即 π_i 就代表从起始点 s 出发到达点 i 的最短距离。

4.5 多商品网络流问题的对偶

之前章节探讨了 SPP 的对偶，接下来看一个更困难的例子，多商品网络流（Multicommodity Network Flow，MCNF）问题的对偶。首先给出该问题的数学模型如下：

$$\min \quad \sum_{(i,j)\in A}\sum_{k} c_{ij}^{k}x_{ij}^{k} \tag{MCNF}$$

$$\sum_{j} x_{ij}^{k} - \sum_{j} x_{ji}^{k} = \begin{cases} d_k, & i = s_k \\ -d_k, & i = t_k \\ 0, & \text{其他} \end{cases} \tag{4.47}$$

$$\sum_{k} x_{ij}^{k} \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{4.48}$$

$$x_{ij}^{k} \geqslant 0, \qquad \forall (i,j) \in A, k \in K \tag{4.49}$$

上述模型中，有几个判断条件，比如 $i = s_k$，另外，变量还是 x_{ij}^{k}，稍微有些复杂。

4.5.1 借助 Excel 和具体小算例写出大规模 MCNF 的对偶

同样，下面以文献（Cappanera and Scaparra，2011）中的算例为例子。具体网络和参数如图 4.6 所示。

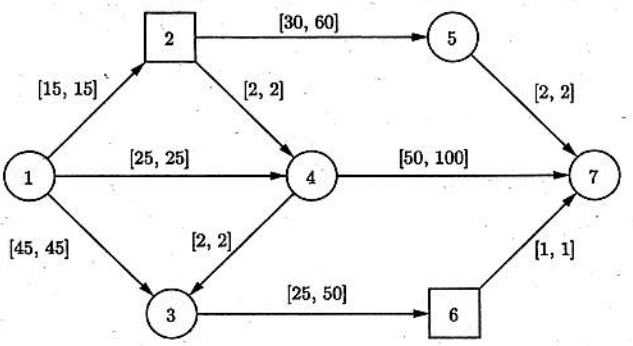


图 4.6 多商品网络流：示例网络

假设这里考虑有 2 个商品流，分别表示如下：

（1）commodity 1：[1, 7, 25]

（2）commodity 2：[2, 6, 2]

其中，[1, 7, 25] 表示起点为 1、终点为 7、需求是 25 单位。

然后按照模型和算例网络结构，把具体模型写出来，如图 4.7所示。

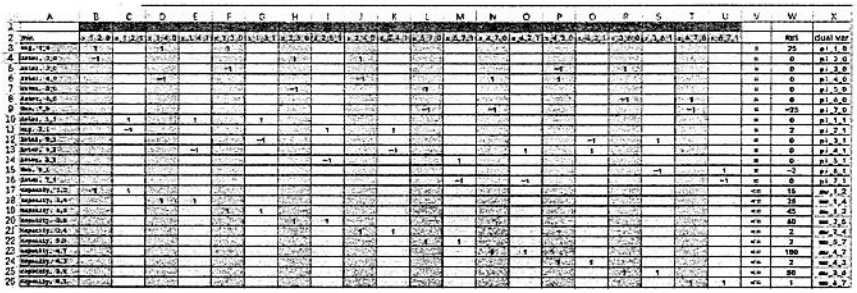


图 4.7　MCNF 模型：表格形式

这里 $x_1_2_0$ 就对应 x_{12}^0，其余变量的对应关系类似。

这个看上去不太有规律，我们按照商品流（commodity）k 把上面的表格整理一下，变成下面更直观的形式。

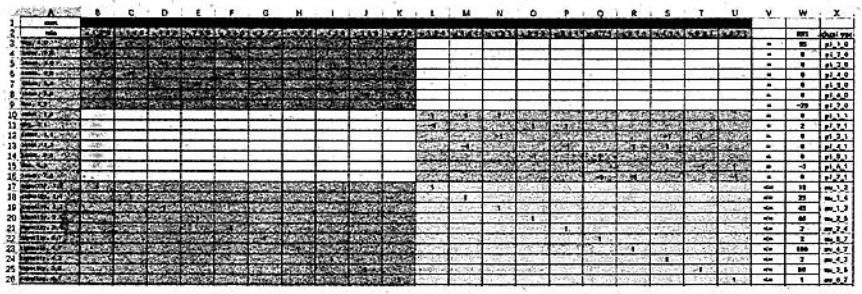


图 4.8　MCNF 模型：表格形式（排序后）

现在看上去就比较清楚了。我们仍然把这个表格叫作原问题表。接下来我们按照同样的方法，根据原问题表生成对偶问题表，如图 4.9所示。（图中第 3 行 Dual 是对偶问题的解，是通过求解原问题得来的。）

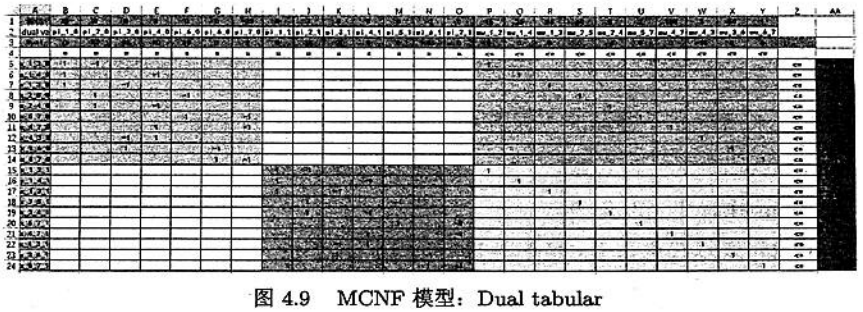


图 4.9　MCNF 模型：Dual tabular

4.5.2 将 Excel 中的对偶问题表转化成公式形式

为了区分 u 和 μ，对表格中的 mu，我们就用 λ 进行替换，如下：

$$\max \quad \sum_{k\in K} d_k\left(\pi^k_{i=s_k}-\pi^k_{i=t_k}\right)+\sum_{(i,j)\in A} u_{ij}\lambda_{ij} \tag{4.50}$$

$$\text{s.t.} \quad \pi^k_i-\pi^k_j+\lambda_{ij}\leqslant c^k_{ij}, \qquad \forall k\in K,\forall (i,j)\in A \tag{4.51}$$

$$\pi^k_i \text{ free}, \qquad \forall k\in K,\forall i\in V \tag{4.52}$$

$$\lambda_{ij}\leqslant 0, \qquad \forall (i,j)\in A \tag{4.53}$$

当然了，按照运筹学学者的习惯，还是跟之前章节 SPP 的对偶中一样做同样的操作如下：

（1）将所有对偶变量 π^k_i 取相反数；

（2）把原约束中 $\pi^k_i-\pi^k_j$ 改成 $\pi^k_j-\pi^k_i$；

（3）将 $\pi^k_{i=s_k}$ 设置成 0，也就是 $\pi^k_{i=s_k}=0$。

这 3 个隐含小动作，很多论文不会明显提及，如果不细心钻研，有可能会非常困惑。最终 MCNF 模型的对偶问题就变成了下面的形式：

$$\max \quad \sum_{k\in K} d_k\pi^k_{i=t_k}+\sum_{(i,j)\in A} u_{ij}\lambda_{ij} \tag{4.54}$$

$$\text{s.t.} \quad \pi^k_j-\pi^k_i+\lambda_{ij}\leqslant c^k_{ij}, \qquad \forall k\in K,\forall (i,j)\in A \tag{4.55}$$

$$\pi^k_{s_k}=0, \tag{4.56}$$

$$\pi^k_i \text{ free}, \qquad \forall k\in K,\forall i\in V \tag{4.57}$$

$$\lambda_{ij}\leqslant 0, \qquad \forall (i,j)\in A \tag{4.58}$$

4.5.3 Python 调用 Gurobi 求解 MCNF

下面给出 Python 调用 Gurobi 求解 MCNF 的代码：

MCNFGurobi.py

```
from gurobipy import *
import pandas as pd
import numpy as np
from pandas import Series, DataFrame
import math

Arcs = {'1,2': [15, 15]     # cost flow
        ,'1,4': [25, 25]
        ,'1,3': [45, 45]
        ,'2,5': [30, 60]
```

```
        ,'2,4': [2, 2]
        ,'5,7': [2, 2]
        ,'4,7': [50, 100]
        ,'4,3': [2, 2]
        ,'3,6': [25, 50]
        ,'6,7': [1, 1]
       }

Nodes = [1, 2, 3, 4, 5, 6, 7]

commodity = [[1, 7, 25],  # s_i, d_i, demand
             [2, 6, 2] ]

model = Model('MultiCommodity')

# add variables
X = {}
for key in Arcs.keys():
    for k in range(len(commodity)):
        key_x = key + ',' + str(k)
        X[key_x] = model.addVar(lb=0
                                ,ub=Arcs[key][1]
                                ,vtype=GRB.CONTINUOUS
                                ,name= 'x_' + key_x
                               )
# add objective function
obj = LinExpr(0)
for key in Arcs.keys():
    for k in range(len(commodity)):
        key_x = key + ',' + str(k)
        obj.addTerms(Arcs[key][0], X[key_x])
model.setObjective(obj, GRB.MINIMIZE)

# constraint 1
for k in range(len(commodity)):
    for i in Nodes:
        lhs = LinExpr(0)
        for key_x in X.keys():
#             nodes = key_x.split(',')
            if(i == (int)(key_x.split(',')[0]) and k == (int)(key_x.split
        (',')[2])):
                lhs.addTerms(1, X[key_x])
```

```
            if(i == (int)(key_x.split(',')[1]) and k == (int)(key_x.split
        (',')[2])):
                lhs.addTerms(-1, X[key_x])
          if(i == commodity[k][0]):
              model.addConstr(lhs == commodity[k][2], name='org_, ' + str(i) +
          '_' + str(k))
          elif(i == commodity[k][1]):
              model.addConstr(lhs == -commodity[k][2], name='des_, ' + str(i)
        + '_' + str(k))
          else:
              model.addConstr(lhs == 0, name='inter_, ' + str(i) + '_' + str
           (k))

# constraint 2
for key in Arcs.keys():
    lhs = LinExpr(0)
    for k in range(len(commodity)):
        key_x = key + ',' + str(k)
        lhs.addTerms(1, X[key_x])
    model.addConstr(lhs <= Arcs[key][1], name = 'capacity_, ' + key)

model.write('Multicommodity_model.lp')
model.optimize()

for var in model.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
dual = model.getAttr("Pi", model.getConstrs())
print('duals = ', dual)
```

求解结果如下：

MCNFGurobiResult.py

```
Solved in 0 iterations and 0.01 seconds
Optimal objective  1.873000000e+03
x_1,2,0          2.0
x_1,4,0          22.0
x_1,3,0          1.0
x_2,5,0          2.0
x_2,4,1          2.0
x_5,7,0          2.0
x_4,7,0          22.0
x_4,3,1          2.0
```

```
x_3,6,0          1.0
x_3,6,1          2.0
x_6,7,0          1.0

duals =  [75.0, 60.0, 30.0, 50.0, 30.0, 5.0, 0.0, 0.0, 10.0, -20.0, 0.0,
       0.0, -45.0, 0.0, 0.0, 0.0, 0.0, 0.0, -8.0, 0.0, 0.0, -18.0, 0.0, 0.0]
```

接下来继续用 Python 调用 Gurobi 建立 MCNF 的对偶问题的模型并求解，来验证模型的正确性。具体代码如下：

MCNFGurobiDualModel.py

```
from gurobipy import *
import pandas as pd
import numpy as np
from pandas import Series, DataFrame
import math

Arcs = {'1,2': [15, 15]    # cost flow
        ,'1,4': [25, 25]
        ,'1,3': [45, 45]
        ,'2,5': [30, 60]
        ,'2,4': [2, 2]
        ,'5,7': [2, 2]
        ,'4,7': [50, 100]
        ,'4,3': [2, 2]
        ,'3,6': [25, 50]
        ,'6,7': [1, 1]
       }

Nodes = [1, 2, 3, 4, 5, 6, 7]

commodity = [[1, 7, 25],  # s_i, d_i, demand
             [2, 6, 2] ]

DualModel = Model('MultiCommodity Dual')

# add variables
pi = {}
lam = {}

for i in Nodes:
    for k in range(len(commodity)):
        key = str(i) + ',' + str(k)
```

```
        pi[key] = DualModel.addVar(lb=-1000
                                   ,ub=1000
                                   ,vtype=GRB.CONTINUOUS
                                   ,name= 'pi_' + str(i) + '_' + str(k)
                                  )
for key in Arcs.keys():
    lam[key] = DualModel.addVar(lb=-10000
                                ,ub=0
                                ,vtype=GRB.CONTINUOUS
                                ,name= 'lam_' + key
                               )

# add objective function
obj = LinExpr(0)
for key in Arcs.keys():
    obj.addTerms(Arcs[key][1], lam[key])
for k in range(len(commodity)):
    for i in Nodes:
        if(i == commodity[k][1]):
            key = str(i) + ',' + str(k)
            obj.addTerms(commodity[k][2], pi[key])

DualModel.setObjective(obj, GRB.MAXIMIZE)

# constraint 1
for k in range(len(commodity)):
    for arc in Arcs.keys():
        i = (int)(arc.split(',')[0])
        j = (int)(arc.split(',')[1])
        key_pi_i = str(i) + ',' + str(k)
        key_pi_j = str(j) + ',' + str(k)
        DualModel.addConstr(pi[key_pi_j] - pi[key_pi_i] + lam[arc] <= Arcs
        [arc][0], name='cons1_, ' + str(i) + '_' + str(j) + '_' + str(k))

# constraint 2 : set pi_sk_k = 0
for k in range(len(commodity)):
    key = str(commodity[k][0]) + ',' + str(k)
    DualModel.addConstr(pi[key] == 0, name='pi_, ' +  str(commodity[k][0]) +
        ',' + str(k))


DualModel.write('Multicommodity_model_Dual.lp')
```

```
DualModel.optimize()

print('obj = ', DualModel.ObjVal)
for var in DualModel.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
dual = DualModel.getAttr("Pi", DualModel.getConstrs())
print('duals = ', dual)
```

求解结果如下：

MCNFGurobiDualResult.py

```
Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    2.8204000e+04   1.133125e+03   0.000000e+00      0s
      16    1.8730000e+03   0.000000e+00   0.000000e+00      0s

Solved in 16 iterations and 0.01 seconds
Optimal objective  1.873000000e+03

# 目标函数 ∑_{k∈K} d_k π^k_{i=t_k} + ∑_{(i,j)∈A} u_ij λ_ij
obj =  1873.0

# π^k_i 的取值
pi_1_0   0.0
pi_1_1   1000.0
pi_2_0   15.0
pi_2_1   0.0
pi_3_0   45.0
pi_3_1   975.0
pi_4_0   25.0
pi_4_1   955.0
pi_5_0   45.0
pi_5_1   -1000.0
pi_6_0   70.0
pi_6_1   1000.0
pi_7_0   75.0
pi_7_1   -1000.0

# λ_ij 的取值
lam_1,2          0.0
lam_1,4          0.0
lam_1,3          0.0
```

```
lam_2,5          0.0
lam_2,4          -953.0
lam_5,7          -28.0
lam_4,7          0.0
lam_4,3          -18.0
lam_3,6          0.0
lam_6,7          -4.0

duals =  [2.0, 22.0, 1.0, 2.0, 0.0, 2.0, 22.0, -0.0, 1.0, 1.0, 0.0, 0.0,
       0.0, 0.0, 2.0, 0.0, 0.0, 2.0, 2.0, 0.0, 25.0, 2.0]
```

可以看到，该解与原模型的解是相同的，且对偶问题的对偶变量，也和原问题的最优解是相同的。

到这里，相信大家已经对上面的方法有了比较全面的理解，在之后的科研工作中遇到类似的问题，也可以更得心应手。

PART TWO

第II部分

常用优化求解器API详解及应用案例

第5章 CPLEX的Java API详解及简单案例

本章主要介绍 Java 调用 CPLEX 的 API，以 Eclipse 作为集成开发环境（IDE）为例，讲解 CPLEX Java 的安装与配置，然后分析建模过程以及其中用到的变量与函数，并通过实例演示帮助读者深入理解。本章大量内容参考自 CPLEX 官方用户手册（参见网址https://www.ibm.com/support/knowledgecenter），为方便阅读起见，我们对具体参考位置不做特殊标注。

5.1 基于 Eclipse 的 CPLEX Java API 的安装与配置：适用于 macOs

CPLEX 官方实例的功能说明参见网址 https://www.ibm.com/support/knowledgecenter/zh/SSSA5P_12.8.0/ilog.odms.cplex.help/CPLEX/Examples/topics/exampleJava.html。

（1）下载安装 CPLEX，并记住安装目录。

（2）导入 Cplex.jar 包。

① 第 1 步如图 5.1 所示。

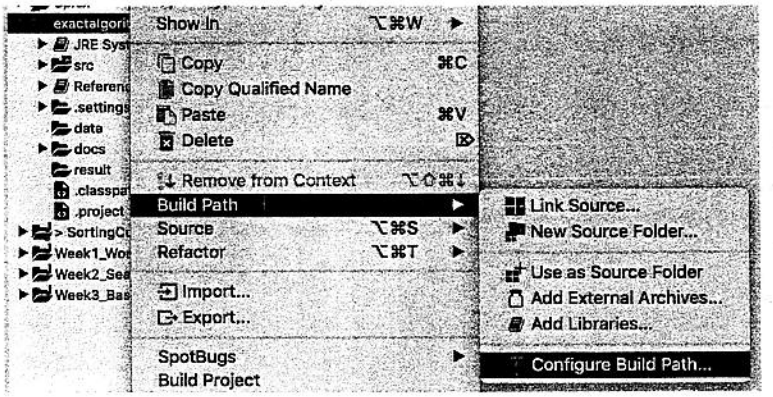


图 5.1 步骤一示意图

② 第 2 步如图 5.2 所示。

③ 第 3 步如图 5.3 所示。

（3）设置 Javadoc。

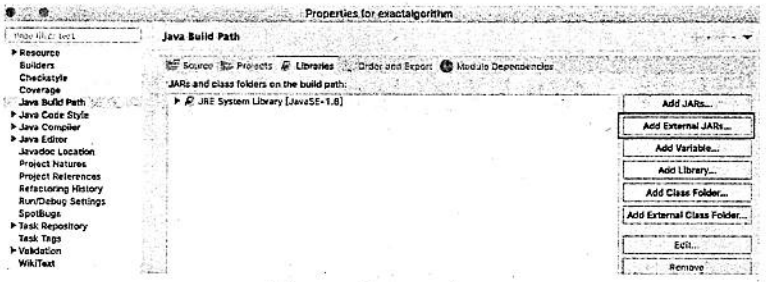


图 5.2 步骤二示意图

① 设置。

第 1 步如图 5.4 所示。

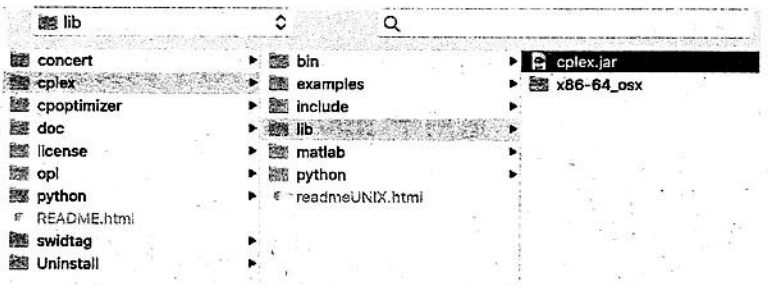


图 5.3 步骤三示意图

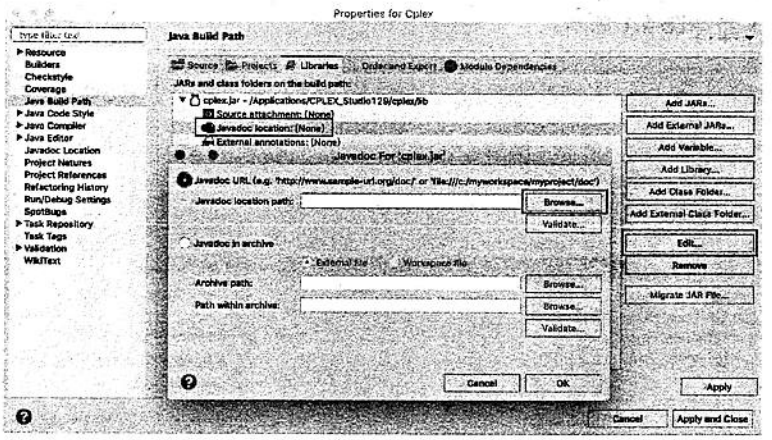


图 5.4 Javadoc 设置第 1 步

第 2 步如图 5.5 所示。

② 效果。

悬停查看类、方法、属性等的说明如图 5.6 所示。

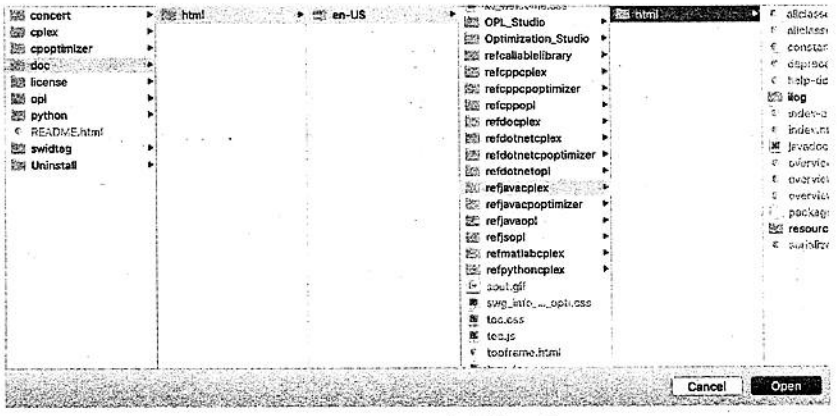


图 5.5 Javadoc 设置第 2 步

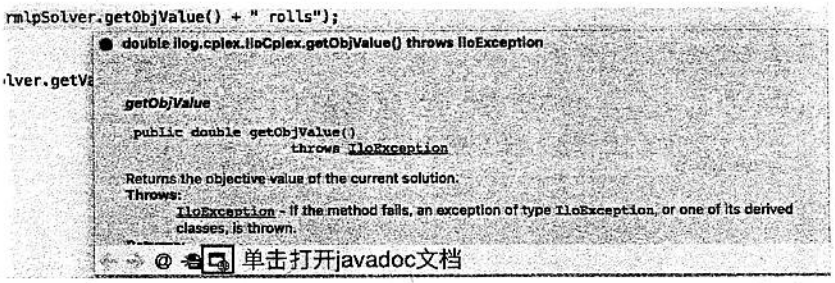


图 5.6 Javadoc 效果 1

在 Eclipse 中直接打开 Javadoc 文件，如图 5.7 所示。

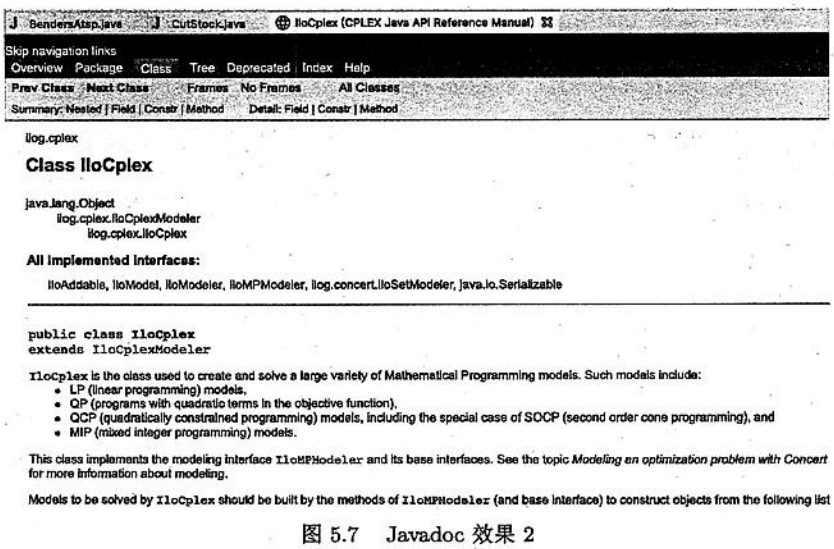


图 5.7 Javadoc 效果 2

（4）设置 Djava.library.path。

该功能是指定依赖的非 Java 库文件路径，比如用来添加指定的 *.so、*.dll。

① 方法 1:设置 JVM 参数,参见网址 https://www.ibm.com/support/knowledgecenter/SSSA5P_12.5.0/ilog.odms.cplex.help/CPLEX/GettingStarted/topics/set_up/Eclipse.html。

② 方法 2：设置 Native Library Location。

第 1 步如图 5.8 所示。

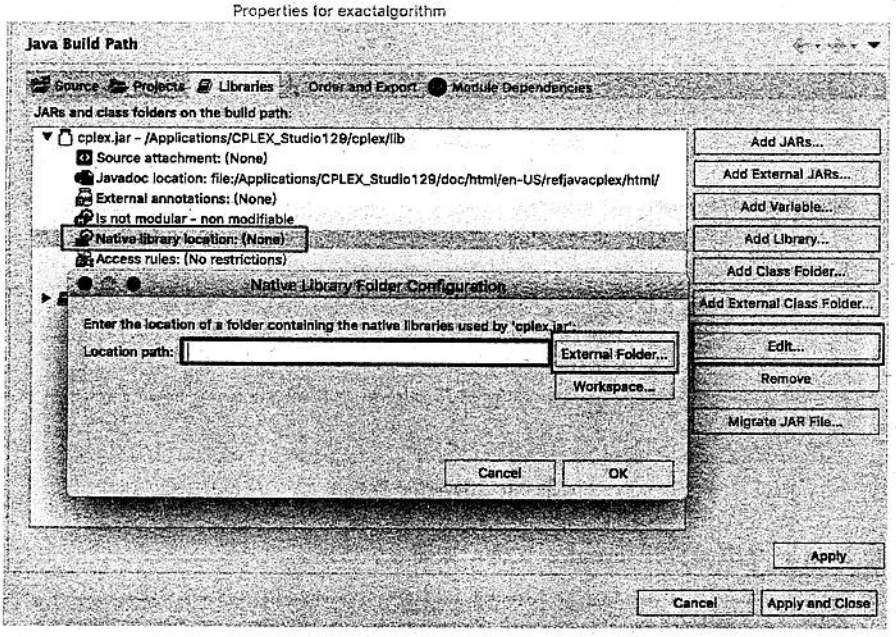


图 5.8 Djava.library.path1

第 2 步如图 5.9 所示。

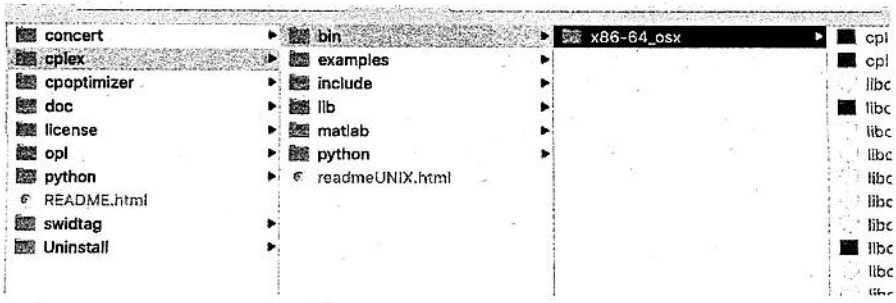


图 5.9 Djava.library.path2

（5）完成，如图 5.10 所示。

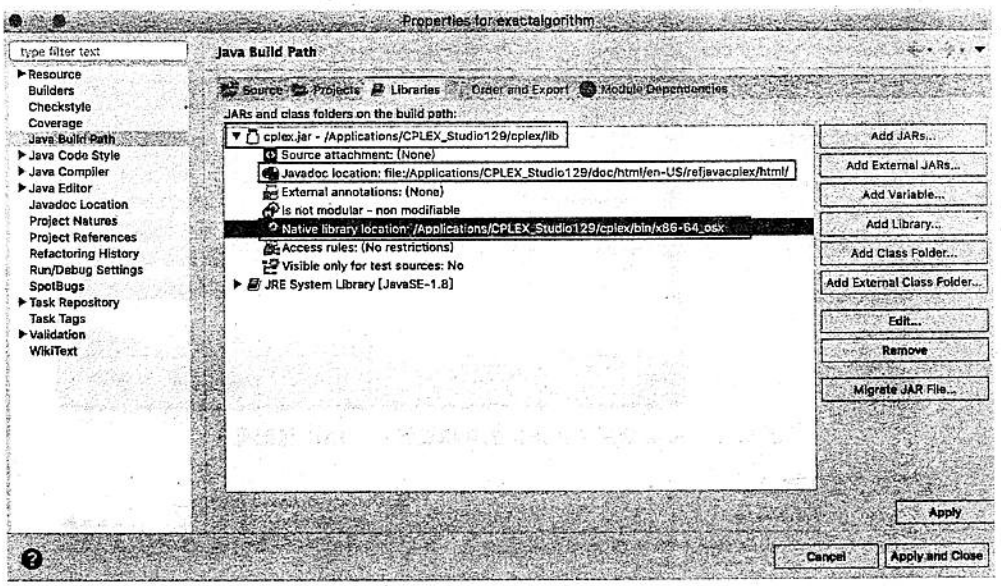


图 5.10 完成

5.2 基于 Eclipse 的 CPLEX Java API 的安装与配置：适用于 Windows

5.2.1 基本环境配置

Java 调用 CPLEX 之前首先需要安装 JDK 和 CPLEX。这里选用比较常用的 Eclipse 作为编辑器，以此实现 Java 调用 CPLEX。

安装完 JDK 和 CPLEX 之后，还需要把 CPLEX 提供的 JAR 包 cplex.jar 添加到 Java 项目的路径中去，才能实现 Java 对 CPLEX 的调用。

具体设置方法如下。

（1）新建一个 Java 的 Project；

（2）右击 Project，选中 Build Path；

（3）选择 Libries -> Add External JARs；

（4）找到 CPLEX 的安装目录下的 C:\Develop\CPLEX128\cplex\lib 目录下的 cplex.jar 即可；

（5）单击 Apply and Close 按钮。

具体实现过程分别如图 5.11 和图 5.12 所示。

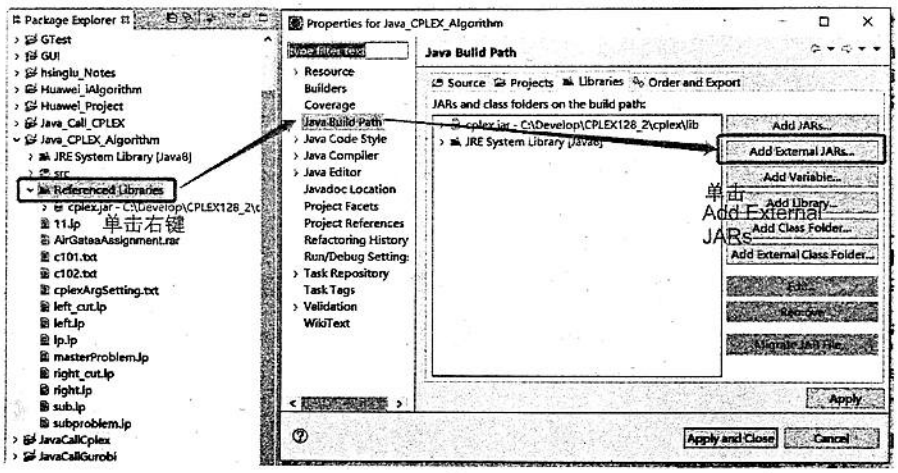


图 5.11 Java 调用 CPLEX 的环境设置——JAR 包配置 1

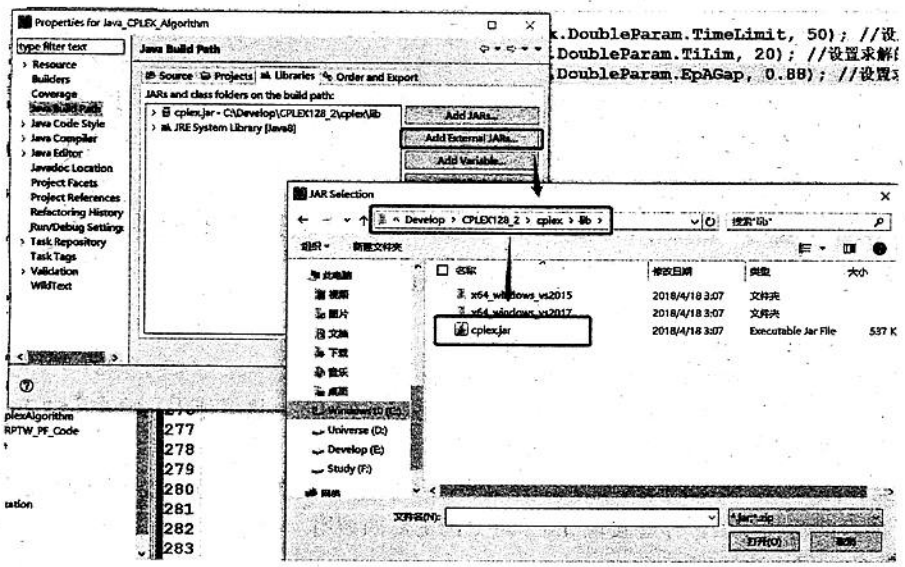


图 5.12 Java 调用 CPLEX 的环境设置——JAR 包配置 2

5.2.2 环境设置中 java.library.path 的问题

完成 JAR 包的设置以后，在运行代码时还有可能会出现如图 5.13 所示的问题。

如果出现以上的报错信息 no cplex in java.library.path，提示说 try invoking java with java -Djava.library.path = ...，我们可以做以下处理：选择 Run -> Run configurations -> Arguments -> VM Arguments，在其中输入-Djava.library.path=C:\Develop\CPLEX128\cplex\bin\x64_win64，然后运行即可。也就是把-Djava.library.path 设置成 CPLEX 的安装目录即可。

具体操作如图 5.14 所示。

```
rollwidth:      17.0
size:           3.0     5.0     9.0
amount:         25.0    20.0    15.0
java.lang.UnsatisfiedLinkError: no cplex1280 in java.library.path
java.library.path must point to the directory containing the CPLEX shared library
try invoking java with java -Djava.library.path=...
Exception in thread "main" java.lang.UnsatisfiedLinkError: ilog.cplex.Cplex.CPXopenCPLEX([I)J
        at ilog.cplex.Cplex.CPXopenCPLEX(Native Method)
        at ilog.cplex.CplexI.init(CplexI.java:6834)
        at ilog.cplex.CplexI.<init>(CplexI.java:785)
        at ilog.cplex.IloCplex.<init>(IloCplex.java:10154)
        at ilog.cplex.IloCplex.<init>(IloCplex.java:10169)
        at ColumnGeneration_Hsinglu.Cutstock_Hsinglu.main(Cutstock_Hsinglu.java:49)
```

图 5.13 运行代码时可能会有的报错信息

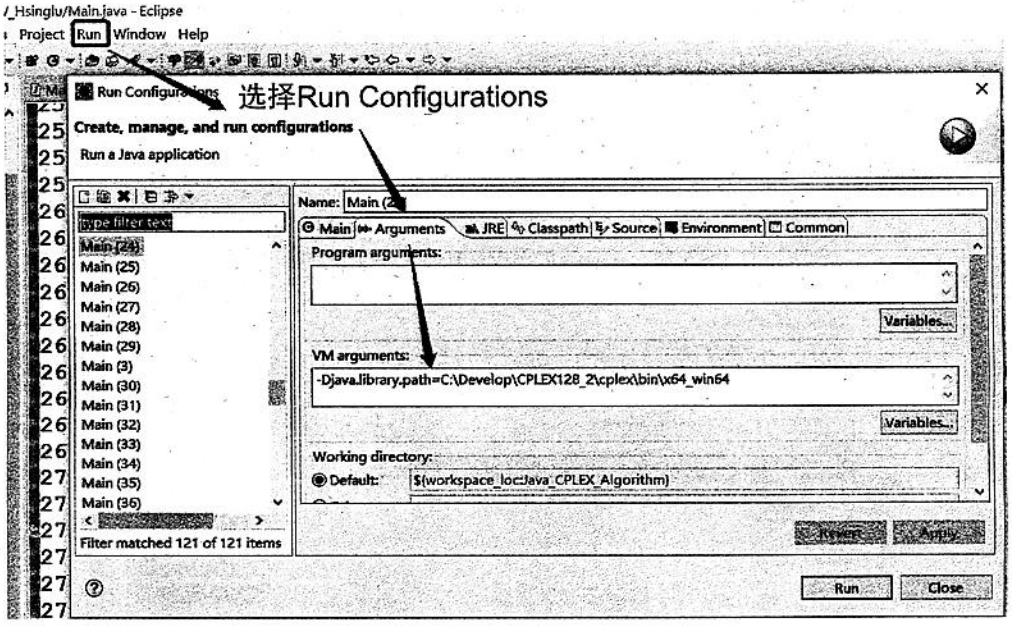


图 5.14 -Djava.library.path 的设置

第 2 种解决办法如下。

(1) 右击 Java Project，选中 Build Path 选项；

(2) 选择 Libries -> cplex.jar 选项；

(3) 打开 cplex.jar 的下拉菜单，选择 Native library location 选项；

(4) 将 Native library location 设置成 C:\Develop\CPLEX128\cplex\bin\x64_win64 即可；

(5) 单击 Apply and Close 按钮。

具体的操作界面分别如图 5.15 和图 5.16 所示。

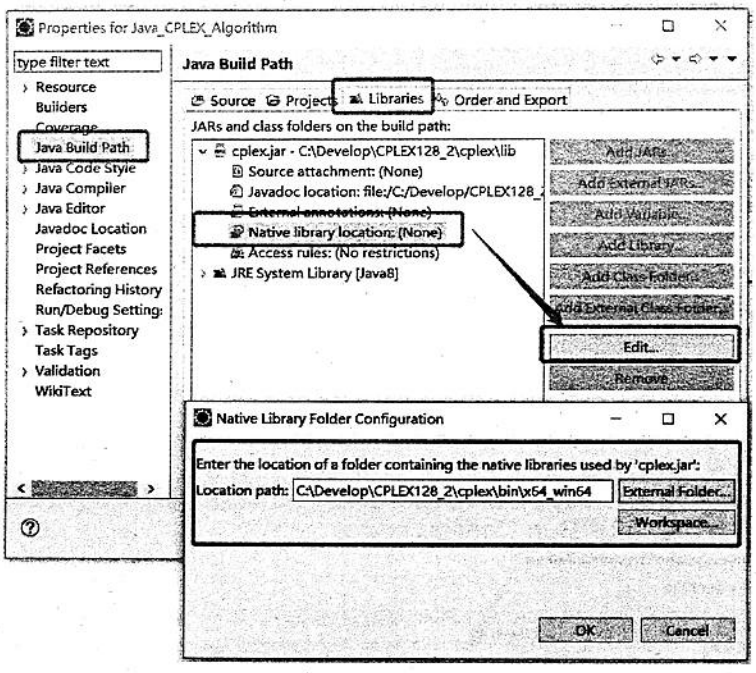


图 5.15 -Djava.library.path 的设置方法 2-1

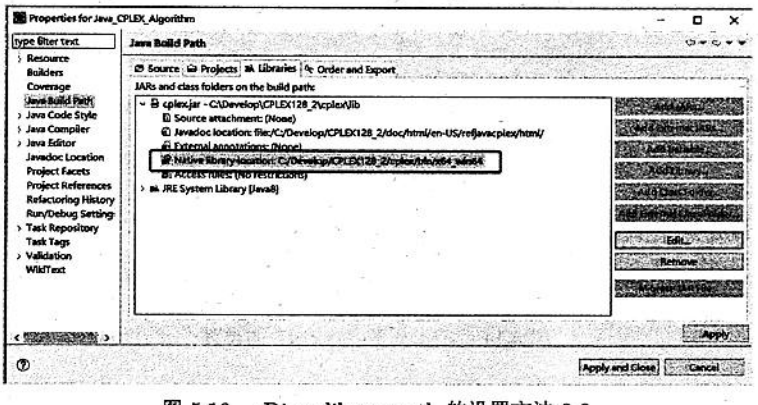


图 5.16 -Djava.library.path 的设置方法 2-2

5.2.3 Javadoc 的设置

为了更好地查看 CPLEX 的 Java 接口相关信息，如类、函数、属性等，我们可以在 Eclipse 中配置 Javadoc。

设置方法如下。

(1) 新建一个 Java 的 Project；

(2) 右击 Project，选中 Build Path 选项；

（3）选择 Libries -> cplex.jar -> 打开下拉可选菜单；

（4）找到 Javadoc location，选择 Javadoc location 选项，单击 Edit 按钮，之后跳出文件系统；

（5）选择 CPLEX 的安装目录下的 C:\Develop\CPLEX128\doc\html\en-US\refjava-cplex\html\；

（6）单击 Apply and Close 按钮。

设置的具体界面分别如图 5.17 和图 5.18 所示。

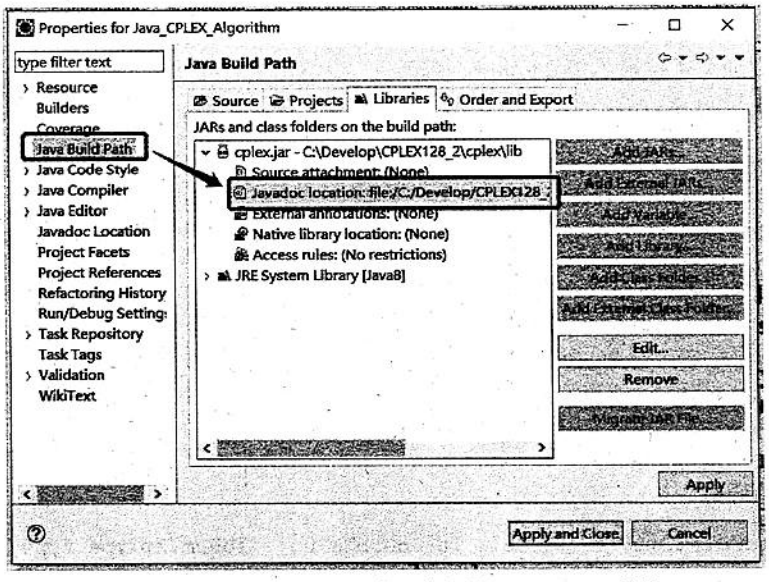


图 5.17 Java 调用 CPLEX 的环境设置——Javadoc 设置 1

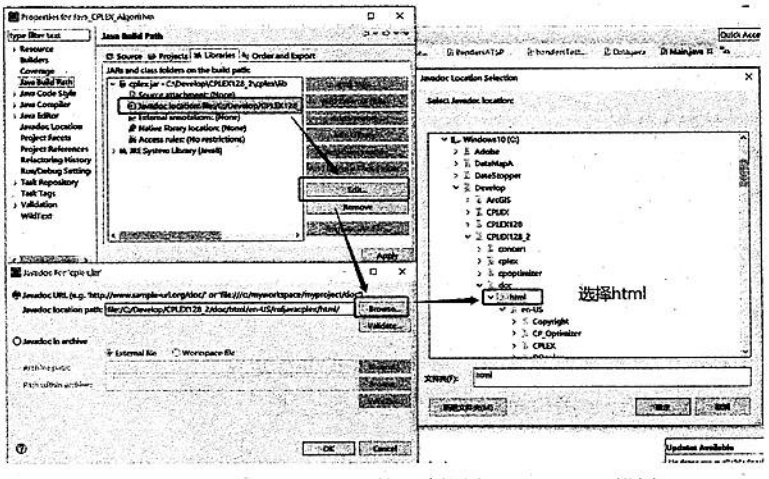


图 5.18 Java 调用 CPLEX 的环境设置——Javadoc 设置 2